

[TÊN TRƯỜNG — TÊN KHOA]

[Logo trường]

BÁO CÁO ĐỒ ÁN TỐT NGHIỆP

QUOCÉ

HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ & QUẢN LÝ DỮ LIỆU SẢN PHẨM (PIM) ENTERPRISE

Giảng viên hướng dẫn: [.....]

Sinh viên thực hiện: [.....]

MSSV: [.....] Lớp: [.....]

[Thành phố], 2026

LỜI CẢM ƠN

[Khung mẫu — người thực hiện tự viết lời cảm ơn tại đây. Gợi ý các ý nên có:]

- Lời cảm ơn Nhà trường, Khoa và quý Thầy/Cô đã truyền đạt kiến thức trong suốt quá trình học tập.
- Lời cảm ơn Giảng viên hướng dẫn — nêu cụ thể sự hỗ trợ, góp ý đã nhận được trong quá trình làm đồ án.
- Lời cảm ơn gia đình, bạn bè đã đồng hành, hỗ trợ trong thời gian thực hiện đồ án.
- Lời cam đoan ngắn gọn về tính trung thực của báo cáo (nếu trường yêu cầu).

MỤC LỤC

LỜI CẢM ƠN	2
MỤC LỤC	3
DANH MỤC HÌNH ẢNH	6
DANH MỤC BẢNG	7
DANH MỤC TỪ VIẾT TẮT	8
CHƯƠNG 1 — TỔNG QUAN	9
1.1 Lý do chọn đề tài.....	9
1.2 Mục tiêu đề tài	9
1.3 Đối tượng và phạm vi	10
1.4 Phương pháp thực hiện	10
1.5 Bố cục báo cáo.....	11
CHƯƠNG 2 — CƠ SỞ LÝ THUYẾT	12
2.1 Backend	12
NestJS 11	12
Prisma ORM 6 + PostgreSQL (Neon Serverless).....	12
Redis (Upstash) qua @nestjs/cache-manager + @keyv/redis	12
JWT + Cookie HttpOnly + Refresh Token Rotation	13
PayOS SDK v2 (@payos/node).....	13
Clouddinary.....	13
Google OAuth2 (google-auth-library)	13
Sentry (@sentry/node) + nestjs-pino	14
@nestjs/throttler + class-validator.....	14
2.2 Frontend	14
Next.js 16 (App Router) + React 19	14
Tailwind CSS	15

axios	15
qrcode.react	15
@react-oauth/google	15
2.3 Nhận xét chung	15
CHƯƠNG 3 — PHÂN TÍCH & THIẾT KẾ HỆ THỐNG	17
3.1 Yêu cầu chức năng và phi chức năng	17
3.1.1 Yêu cầu chức năng	17
3.1.2 Yêu cầu phi chức năng	17
3.2 Sơ đồ Use Case	17
3.3 Thiết kế cơ sở dữ liệu	18
3.4 Kiến trúc hệ thống	20
3.5 Các luồng nghiệp vụ quan trọng	21
3.5.1 Đăng ký / Đăng nhập và cơ chế Access — Refresh Token	21
3.5.2 Đặt hàng và thanh toán VietQR — điểm nhấn kỹ thuật của đồ án	22
3.5.3 Vòng đời trạng thái đơn hàng	25
3.5.4 Luồng bảo mật nhiều lớp	26
3.5.5 Cấu trúc thư mục lưu trữ ảnh trên Cloudinary	28
CHƯƠNG 4 — XÂY DỰNG & TRIỂN KHAI	29
4.1 Môi trường phát triển	29
4.2 Chi tiết các module đã xây dựng	29
4.2.1 Auth	29
4.2.2 Users	29
4.2.3 Product / MDM (Categories, SubCategories, Brands)	29
4.2.4 Orders / Payment	29
4.2.5 Discounts	30
4.2.6 AuditLog	30
4.2.7 Reviews	30

4.2.8 Addresses	30
4.2.9 Cloudinary / AI	30
4.2.10 Health	30
4.3 Một số đoạn code tiêu biểu	30
4.3.1 Đọc JWT từ Cookie HttpOnly (JwtStrategy)	31
4.3.2 Chống race condition khi áp dụng mã giảm giá	31
4.3.3 Xử lý Webhook thanh toán PayOS	31
4.3.4 Cập nhật biến thể sản phẩm: xoá sạch rồi tạo lại	32
4.3.5 Chỉ hoàn kho khi kho đã thực sự bị trừ (fix ngày 26/09/2026)	33
4.4 Giao diện chương trình	33
4.4.1 Storefront — Khách hàng	34
4.4.2 Tài khoản khách hàng	39
4.4.3 Trang quản trị (Admin)	40
CHƯƠNG 5 — KIỂM THỬ HỆ THỐNG	45
5.1 Kế hoạch kiểm thử	45
5.2 Kết quả kiểm thử	45
5.2.1 Danh sách lỗi thật đã phát hiện và khắc phục	47
5.3 Nhận xét về chất lượng kiểm thử	48
CHƯƠNG 6 — KẾT LUẬN & HƯỚNG PHÁT TRIỂN	49
6.1 Kết quả đạt được	49
6.2 Hạn chế	49
6.3 Hướng phát triển	50
TÀI LIỆU THAM KHẢO	52
PHỤ LỤC A — DANH SÁCH ĐẦY ĐỦ API ENDPOINT	53

DANH MỤC HÌNH ẢNH

<i>Sơ đồ Use Case theo vai trò</i>	18
<i>Sơ đồ thực thể quan hệ (ERD)</i>	19
<i>Kiến trúc hệ thống tổng quan</i>	21
<i>Luồng đăng ký/đăng nhập và cơ chế Access/Refresh Token</i>	22
<i>Luồng đặt hàng và thanh toán VietQR qua PayOS</i>	24
<i>Sơ đồ trạng thái vòng đời đơn hàng</i>	26
<i>Luồng bảo mật nhiều lớp minh hoạ qua PATCH /products/:id</i>	27
<i>Cấu trúc thư mục lưu ảnh trên Cloudinary</i>	28
<i>Trang chủ storefront</i>	35
<i>Trang chi tiết sản phẩm (chọn biến thể màu, đánh giá)</i>	35
<i>Trang đăng nhập (email và Google OAuth2)</i>	36
<i>Giỏ hàng — chọn từng dòng để thanh toán một phần</i>	36
<i>Trang thanh toán — chọn phương thức COD/VietQR, progress-step</i>	37
<i>Tra cứu đơn hàng dành cho khách vãng lai</i>	37
<i>Trang Điều khoản dịch vụ</i>	38
<i>Lịch sử đơn hàng cá nhân</i>	39
<i>Sổ địa chỉ giao hàng</i>	40
<i>Trang quản lý sản phẩm — phân trang, tìm kiếm, lọc theo danh mục/thương hiệu</i>	41
<i>Form thêm sản phẩm mới — SKU tự sinh, danh mục/thương hiệu chọn nhanh</i>	42
<i>Trang quản lý đơn hàng — trạng thái theo state machine</i>	43
<i>Modal chi tiết đơn hàng và các nút chuyển trạng thái hợp lệ</i>	44

DANH MỤC BẢNG

<i>Mô tả các thực thể chính trong cơ sở dữ liệu.....</i>	<i>19</i>
<i>Tổng hợp một số kết quả kiểm thử tiêu biểu (số liệu thật, trích từ nhật ký phát triển) .</i>	<i>45</i>
<i>Các lỗi thật đã phát hiện và khắc phục trong quá trình phát triển</i>	<i>47</i>

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
API	Application Programming Interface — giao diện lập trình ứng dụng
CRUD	Create, Read, Update, Delete — 4 thao tác cơ bản trên dữ liệu
DTO	Data Transfer Object — đối tượng dùng để validate/truyền dữ liệu request
ERD	Entity Relationship Diagram — sơ đồ thực thể quan hệ
FK	Foreign Key — khoá ngoại
HMAC	Hash-based Message Authentication Code — mã xác thực dựa trên hàm băm
JWT	JSON Web Token — chuẩn token dùng cho xác thực
MDM	Master Data Management — quản lý dữ liệu chủ (danh mục, thương hiệu...)
ORM	Object-Relational Mapping — ánh xạ đối tượng - quan hệ (Prisma)
PIM	Product Information Management — quản lý thông tin sản phẩm
RBAC	Role-Based Access Control — kiểm soát truy cập theo vai trò
SDK	Software Development Kit — bộ công cụ phát triển phần mềm
SEO	Search Engine Optimization — tối ưu hoá công cụ tìm kiếm
SKU	Stock Keeping Unit — mã quản lý kho của 1 mặt hàng
TTL	Time To Live — thời gian sống của 1 bản ghi cache
UUID	Universally Unique Identifier — định danh duy nhất toàn cục
VietQR	Chuẩn mã QR thanh toán liên ngân hàng tại Việt Nam

CHƯƠNG 1 — TỔNG QUAN

1.1 Lý do chọn đề tài

Thương mại điện tử tại Việt Nam tiếp tục tăng trưởng mạnh trong những năm gần đây, kéo theo nhu cầu thực tế về các hệ thống bán hàng trực tuyến vừa đủ đầy đủ nghiệp vụ (giỏ hàng, thanh toán, vận đơn, khuyến mãi) vừa phải xử lý tốt bài toán quản lý dữ liệu sản phẩm (Product Information Management — PIM) khi danh mục hàng hoá có nhiều danh mục, thương hiệu và biến thể màu sắc khác nhau. Phần lớn đồ án sinh viên ở dạng CRUD đơn giản, chưa chạm tới các vấn đề kỹ thuật thật sự phát sinh khi vận hành một hệ thống thương mại điện tử: tính toàn vẹn dữ liệu khi thanh toán bất đồng bộ qua webhook, chống trừ kho sai lệch khi có nhiều luồng xử lý đồng thời, hay việc chuẩn hoá quyền truy cập theo nhiều vai trò khác nhau.

Đề tài QUOCÉ được thực hiện nhằm xây dựng một hệ thống thương mại điện tử kết hợp PIM ở mức độ tiệm cận thực tế doanh nghiệp vừa và nhỏ, đồng thời lấy đó làm cơ sở để tìm hiểu và áp dụng các nguyên tắc bảo mật, thiết kế API và xử lý giao dịch thanh toán đúng chuẩn — thay vì chỉ dừng lại ở mức minh hoạ giao diện.

1.2 Mục tiêu đề tài

- Xây dựng backend RESTful API bằng NestJS + Prisma + PostgreSQL, áp dụng đầy đủ các lớp bảo mật: xác thực JWT qua cookie HttpOnly, phân quyền theo vai trò (RBAC), giới hạn tần suất request, validate dữ liệu đầu vào nghiêm ngặt.
- Xây dựng module PIM thực sự có quan hệ dữ liệu chuẩn hoá (Category — SubCategory — Brand — Product — ProductVariant) thay vì lưu chuỗi tự do.
- Tích hợp thanh toán VietQR thật qua PayOS, xử lý đúng vòng đời một giao dịch bất đồng bộ (tạo đơn → hiển thị QR → xác nhận qua webhook → cập nhật kho và trạng thái).
- Xây dựng frontend Next.js hoàn chỉnh cho cả 2 nhóm người dùng: khách hàng (storefront) và quản trị viên (trang quản trị sản phẩm, đơn hàng).
- Ghi nhận trung thực toàn bộ quá trình kiểm thử, các lỗi thật đã phát hiện và cách khắc phục — coi đây là một phần giá trị học thuật của đồ án, không chỉ là bản liệt kê tính năng.

1.3 Đối tượng và phạm vi

Hệ thống phục vụ 3 nhóm đối tượng sử dụng chính (chi tiết xem sơ đồ Use Case ở Chương 3):

- Khách vãng lai / Khách hàng: duyệt sản phẩm, đặt hàng (COD hoặc VietQR) không bắt buộc đăng nhập, tra cứu đơn hàng bằng số điện thoại, và khi đã đăng nhập có thêm chức năng xem lịch sử đơn hàng, đánh giá sản phẩm đã mua, quản lý sổ địa chỉ.
- Quản trị viên (ADMIN): quản lý sản phẩm/biến thể, danh mục, thương hiệu, đơn hàng theo quy trình trạng thái (state machine), xem cảnh báo tồn kho thấp.
- Đối tác bán hàng (VENDOR): vai trò tồn tại trong hệ thống phân quyền nhưng phạm vi hiện tại CHỈ có quyền upload ảnh — hệ thống chưa xây dựng luồng nghiệp vụ riêng cho VENDOR (được ghi rõ trong phần Hạn chế ở Chương 6, không che giấu).

Phạm vi đồ án dừng lại ở môi trường phát triển (development) đã được kiểm thử kỹ qua công cụ dòng lệnh, script kiểm thử tự viết và kiểm thử giao diện thủ công; hệ thống CHƯA được triển khai lên môi trường production thật với tên miền và SSL riêng — đây là hạn chế được nêu rõ, không phải điểm bị bỏ sót.

1.4 Phương pháp thực hiện

Đồ án được thực hiện theo phương pháp lặp (iterative) chia theo từng nhóm chức năng độc lập thay vì mô hình thác nước một lần: bắt đầu từ nhóm và lỗi bảo mật khẩn cấp trên phần khung có sẵn (Phase 0), sau đó triển khai từng nhóm — chuẩn hoá dữ liệu chủ (MDM), quan sát hệ thống (logging/health-check/Sentry), sẵn sàng chịu tải (cache, phân trang), hoàn thiện nghiệp vụ (mã giảm giá, nhật ký thao tác), rồi mới tới hoàn thiện giao diện. Mỗi nhóm việc đều được tự kiểm thử bằng script gọi API thật hoặc kiểm thử giao diện thủ công trước khi chuyển sang nhóm tiếp theo, và toàn bộ quá trình — kể cả các lỗi phát sinh và cách sửa — được ghi log chi tiết theo thời gian thực trong suốt quá trình phát triển.

Một điểm cần nêu trung thực về phương pháp: quá trình phát triển có sử dụng trợ lý AI lập trình cặp (AI pair-programming) làm công cụ hỗ trợ viết code và tra cứu tài liệu kỹ

thuật dưới sự giám sát, kiểm tra và quyết định của người thực hiện ở mọi bước quan trọng (thiết kế schema, quyết định nghiệp vụ, xác nhận kết quả kiểm thử thật trước khi chuyển bước tiếp theo). Toàn bộ quyết định kiến trúc, việc xác nhận kiểm thử qua các công cụ thật (Postman, curl, PowerShell, ngrok cho webhook thật) đều do người thực hiện trực tiếp chạy và xác nhận kết quả trước khi ghi nhận là hoàn thành.

1.5 Bố cục báo cáo

- Chương 1 — Tổng quan: giới thiệu đề tài, mục tiêu, phạm vi, phương pháp thực hiện.
- Chương 2 — Cơ sở lý thuyết: giới thiệu các công nghệ thật sự được sử dụng và lý do lựa chọn.
- Chương 3 — Phân tích & thiết kế hệ thống: yêu cầu, sơ đồ Use Case, ERD, kiến trúc, các luồng nghiệp vụ quan trọng kèm lý giải quyết định thiết kế.
- Chương 4 — Xây dựng & triển khai: chi tiết từng module, các đoạn code tiêu biểu, giao diện chương trình thật.
- Chương 5 — Kiểm thử hệ thống: kế hoạch và kết quả kiểm thử thật, danh sách lỗi đã phát hiện và khắc phục.
- Chương 6 — Kết luận & hướng phát triển: kết quả đạt được, hạn chế trung thực, hướng phát triển tiếp theo.

CHƯƠNG 2 — CƠ SỞ LÝ THUYẾT

Chương này giới thiệu các công nghệ THẬT SỰ được sử dụng trong hệ thống (đối chiếu trực tiếp với `package.json` của cả 2 phía backend/frontend), kèm lý do lựa chọn dựa trên trải nghiệm thực tế trong quá trình xây dựng, không liệt kê công nghệ chưa được dùng.

2.1 Backend

NestJS 11

Framework Node.js theo kiến trúc module hoá (Controller — Service — Module), hỗ trợ Dependency Injection và Decorator, xây dựng trên nền Express.

Lý do lựa chọn: Cấu trúc module rõ ràng giúp tách biệt từng nghiệp vụ (Auth, Product, Orders, Payment...) thành các module độc lập, dễ áp dụng Guard/Interceptor/Pipe dùng chung (ValidationPipe, AuditLogInterceptor, GlobalExceptionHandler) cho toàn bộ ứng dụng mà không phải lặp lại code ở từng route.

Prisma ORM 6 + PostgreSQL (Neon Serverless)

Prisma là ORM thế hệ mới cho TypeScript, sinh Prisma Client type-safe từ file `schema.prisma`. Neon là dịch vụ PostgreSQL serverless (tự "ngủ" khi không hoạt động, giúp tiết kiệm chi phí ở quy mô đồ án).

Lý do lựa chọn: Prisma Client tự sinh type TypeScript khớp 100% với schema, giảm hẳn lỗi runtime do sai tên cột/kiểu dữ liệu. `\$transaction` của Prisma được dùng xuyên suốt cho các nghiệp vụ đòi hỏi tính nguyên tử (tạo đơn hàng, xử lý webhook thanh toán) — chi tiết ở Chương 3.

Redis (Upstash) qua @nestjs/cache-manager + @keyv/redis

Redis là cơ sở dữ liệu key-value trong bộ nhớ, dùng làm tầng cache. Upstash cung cấp Redis serverless miễn phí phù hợp quy mô đồ án.

Lý do lựa chọn: Package `cache-manager-redis-yet` (thường gặp trong hướng dẫn cũ) đã ngừng phát triển; `@keyv/redis` là driver được `@nestjs/cache-manager` v12 khuyến nghị hiện tại. Cache được áp dụng có chọn lọc cho danh mục/thương hiệu (dữ liệu ít

thay đổi, TTL 1 giờ), KHÔNG áp dụng cho danh sách sản phẩm phân trang vì số tổ hợp tham số lọc là vô hạn và Redis qua Keyv không hỗ trợ xoá theo prefix để invalidate đúng.

JWT + Cookie HttpOnly + Refresh Token Rotation

JSON Web Token dùng để xác thực không trạng thái (stateless); cookie HttpOnly là cookie mà JavaScript phía trình duyệt không đọc được, chỉ trình duyệt tự gửi kèm request.

Lý do lựa chọn: Lưu accessToken trong `localStorage` (cách làm phổ biến trong nhiều hướng dẫn) dễ bị đánh cắp qua tấn công XSS. Chuyển sang cookie HttpOnly loại bỏ hẳn nguy cơ này. accessToken có vòng đời ngắn (15 phút) để giảm thiệt hại nếu bị lộ; refreshToken (30 ngày) được xoay vòng (rotation) và chỉ lưu bản băm SHA-256 trong DB — theo đúng khuyến nghị không lưu token dạng plaintext.

PayOS SDK v2 (@payos/node)

PayOS là cổng thanh toán VietQR (chuyển khoản ngân hàng qua mã QR) phổ biến tại Việt Nam, cung cấp SDK Node.js chính thức.

Lý do lựa chọn: PayOS được chọn vì hỗ trợ VietQR — phương thức thanh toán phổ biến, không cần tài khoản merchant quốc tế phức tạp như Stripe/PayPal. Bài học thực tế quan trọng: API của package đổi hoàn toàn giữa v1 và v2 (`createPaymentLink` → `paymentRequests.create`, `verifyPaymentWebhookData` → `webhooks.verify`) — quá trình xây dựng đã phải xác nhận lại đúng API thật đang cài (qua `npm list` và đọc trực tiếp type definition trong `node\_modules`) thay vì suy đoán từ kiến thức đã biết, tránh lặp lại nhiều giờ debug từng gặp phải.

Cloudinary

Dịch vụ lưu trữ và xử lý ảnh trên đám mây, hỗ trợ transformation (resize, tối ưu định dạng) ngay khi upload.

Lý do lựa chọn: Miễn phí ở quy mô nhỏ, API upload đơn giản qua stream, hỗ trợ transformation `quality:auto, fetch\_format:auto` giúp tối ưu dung lượng ảnh sản phẩm tự động mà không cần xử lý ảnh thủ công.

Google OAuth2 (google-auth-library)

Thư viện chính thức của Google để xác thực ID Token nhận được từ luồng đăng nhập Google phía client.

Lý do lựa chọn: Cho phép người dùng đăng nhập nhanh không cần tạo mật khẩu riêng. Điểm mấu chốt về bảo mật: ID Token PHẢI được xác minh chữ ký bằng `'OAuth2Client.verifyIdToken()'` ở phía server — không bao giờ tin trực tiếp email/tên do client tự gửi kèm (bài học từ 1 lỗ hổng thật đã phát hiện và sửa, xem Chương 5).

Sentry (@sentry/node) + nestjs-pino

Sentry là dịch vụ theo dõi lỗi (error tracking) tập trung; nestjs-pino tích hợp Pino — thư viện ghi log JSON hiệu năng cao — vào vòng đời request của NestJS.

Lý do lựa chọn: Pino ghi log có cấu trúc (structured JSON) giúp dễ tra cứu và tự động ẩn (redact) các trường nhạy cảm như mật khẩu, token, cookie khỏi log. Sentry được khởi tạo nhằm mục tiêu gom lỗi 500 thật về một nơi theo dõi tập trung — mức độ tích hợp thực tế hiện tại được nêu rõ ở Chương 5/6 (đã khởi tạo nhưng CHƯA nối vào bộ lọc lỗi toàn cục để thực sự gửi lỗi đi, ghi nhận trung thực đây là hạn chế còn tồn đọng).

@nestjs/throttler + class-validator

Throttler giới hạn số lượng request trong một khoảng thời gian theo IP; class-validator cho phép khai báo ràng buộc validate ngay trên class DTO bằng decorator.

Lý do lựa chọn: Áp dụng giới hạn riêng cho từng endpoint nhạy cảm (đăng nhập, đăng ký, tra cứu đơn, chat AI) để giảm nguy cơ dò mật khẩu/quét dữ liệu. `'ValidationPipe'` toàn cục với `'whitelist'` và `'forbidNonWhitelisted'` từ chối thẳng mọi field lạ trong request thay vì âm thầm bỏ qua, giúp phát hiện sớm client gửi sai định dạng.

2.2 Frontend

Next.js 16 (App Router) + React 19

Next.js là framework React hỗ trợ định tuyến theo cấu trúc thư mục (App Router) và nhiều chế độ render.

Lý do lựa chọn: App Router là kiến trúc định tuyến hiện hành của Next.js, phù hợp xây dựng cả trang công khai (storefront) lẫn khu vực quản trị trong cùng 1 dự án. Phần lớn trang trong hệ thống dùng Client Component vì cần tương tác nhiều (giỏ hàng, form,

polling trạng thái thanh toán); 3 trang chính sách pháp lý dùng Server Component vì nội dung tĩnh.

Tailwind CSS

Framework CSS tiện ích (utility-first), styling trực tiếp qua class trong JSX.

Lý do lựa chọn: Tốc độ triển khai giao diện nhanh, dễ đảm bảo tính nhất quán khi đã thống nhất 1 bộ token thiết kế (bo góc vuông, chữ hoa đậm, tương phản đen trắng) áp dụng xuyên suốt toàn site.

axios

Thư viện HTTP client cho JavaScript, hỗ trợ interceptor xử lý request/response tập trung.

Lý do lựa chọn: Cấu hình `withCredentials: true` để trình duyệt tự gửi kèm cookie HttpOnly sang backend (khác origin do khác cổng). Request interceptor tự động phát hiện lỗi 401, gọi `/auth/refresh` để xoay vòng token và phát lại request gốc — người dùng không bị đăng xuất đột ngột giữa phiên làm việc.

qrcode.react

Thư viện React vẽ mã QR từ một chuỗi dữ liệu bất kỳ.

Lý do lựa chọn: PayOS trả về chuỗi VietQR theo chuẩn EMVCo (chuỗi text, không phải URL ảnh) — cần tự vẽ mã QR ở phía client bằng `<QRCodeSVG value={qrCode}>` thay vì hiển thị trực tiếp như một ảnh.

@react-oauth/google

Thư viện React bọc sẵn nút đăng nhập Google và trả về ID Token sau khi người dùng xác thực thành công.

Lý do lựa chọn: Giảm khối lượng code cần tự triển khai luồng OAuth2 phía client, chỉ cần gửi `credential` (ID Token) nhận được lên backend để xác minh.

2.3 Nhận xét chung

Bộ công nghệ được lựa chọn đều là các công cụ phổ biến, có tài liệu chính thức đầy đủ và phù hợp quy mô một hệ thống thương mại điện tử vừa và nhỏ. Điểm chung xuyên

suốt là ưu tiên các cơ chế AN TOÀN MẶC ĐỊNH (cookie HttpOnly thay vì localStorage, ORM type-safe thay vì viết SQL tay, xác minh chữ ký thay vì tin dữ liệu client) — đây cũng là kim chỉ nam xuyên suốt các quyết định thiết kế trình bày ở Chương 3.

CHƯƠNG 3 — PHÂN TÍCH & THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu chức năng và phi chức năng

3.1.1 Yêu cầu chức năng

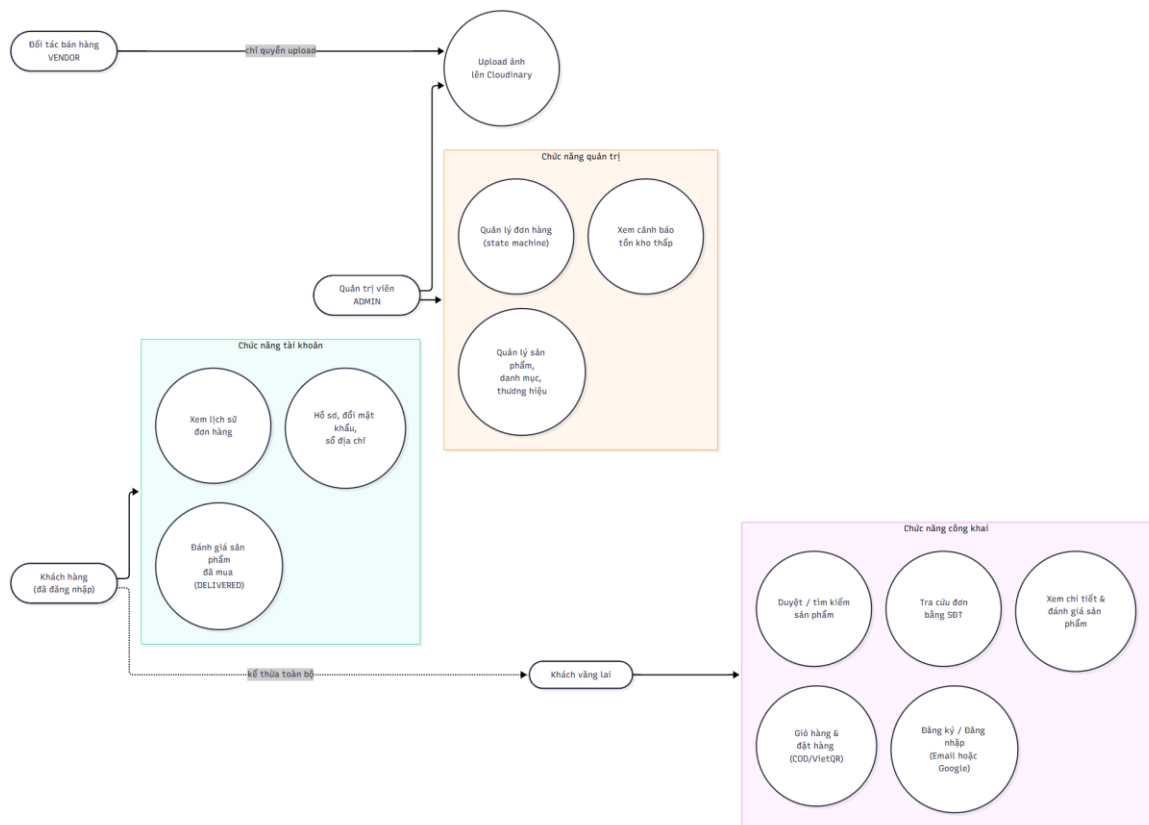
- Quản lý sản phẩm theo cấu trúc phân cấp Category → SubCategory, gắn Brand, hỗ trợ nhiều biến thể màu với giá/tồn kho/ảnh riêng.
- Giỏ hàng phía client, cho phép chọn từng dòng để thanh toán một phần, áp dụng mã giảm giá.
- Đặt hàng theo 2 phương thức: COD (thanh toán khi nhận hàng) và VietQR (qua PayOS), hỗ trợ cả khách vắng lai lẫn khách đã đăng nhập.
- Quản lý vòng đời đơn hàng theo trạng thái vận chuyển (PENDING → PROCESSING → SHIPPED → DELIVERED, có nhánh CANCELLED).
- Đánh giá sản phẩm (chỉ khách đã mua và đã nhận hàng), sổ địa chỉ, tra cứu đơn hàng cho khách vắng lai.
- Trang quản trị cho phép quản lý sản phẩm/đơn hàng, xem cảnh báo tồn kho thấp, ghi nhật ký thao tác nhạy cảm.

3.1.2 Yêu cầu phi chức năng

- Bảo mật: xác thực JWT qua cookie HttpOnly, phân quyền RBAC, giới hạn tần suất request, validate nghiêm ngặt đầu vào, không tin bất kỳ giá trị nhạy cảm nào (giá tiền, quyền sở hữu) do client tự khai.
- Toàn vẹn dữ liệu: mọi thao tác nhiều bước ảnh hưởng tồn kho/tiền đều chạy trong 1 transaction cơ sở dữ liệu.
- Khả năng quan sát (observability): log có cấu trúc, health-check, theo dõi lỗi tập trung.
- Hiệu năng: cache cho dữ liệu ít thay đổi, phân trang cho danh sách lớn.

3.2 Sơ đồ Use Case

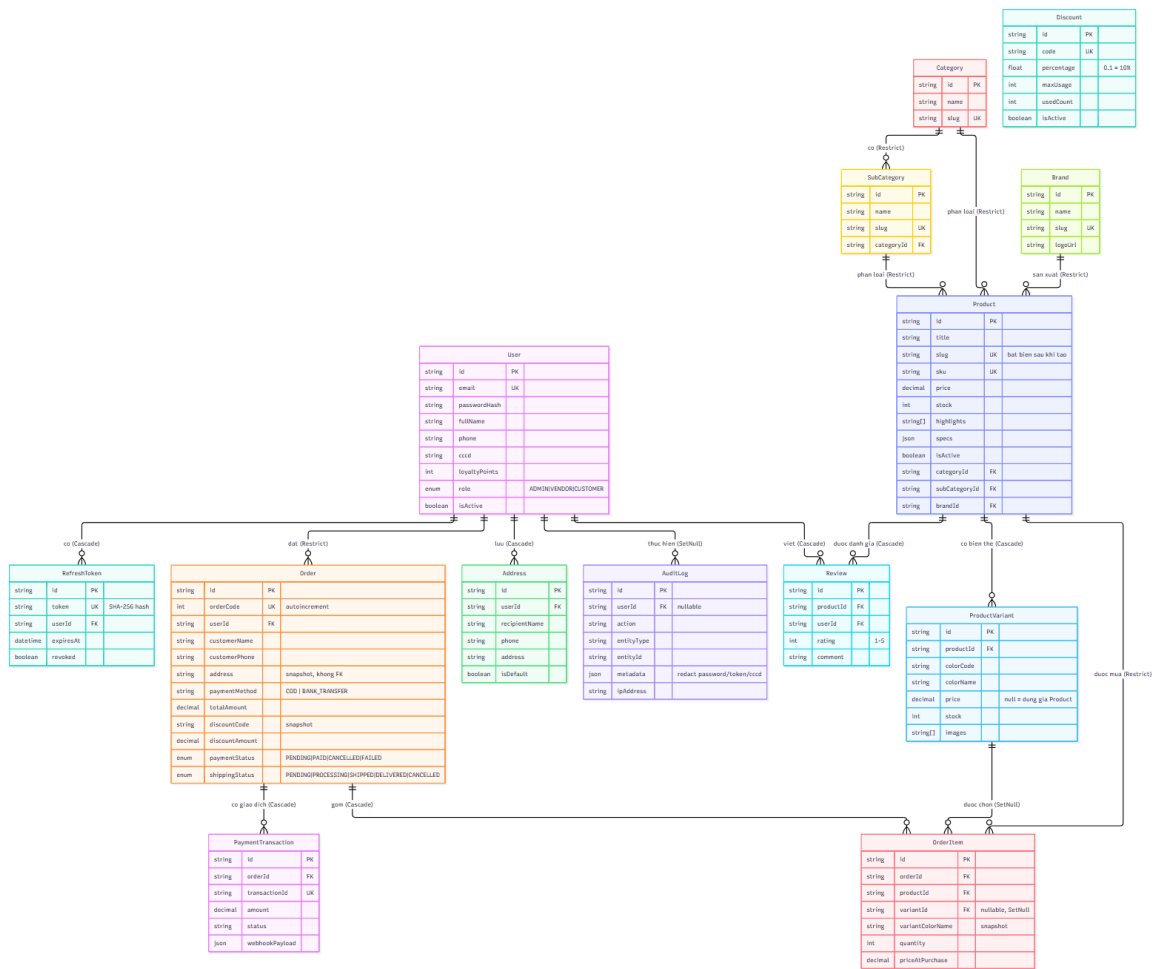
Hệ thống phân chia rõ chức năng theo 3 vai trò: Khách (vắng lai/đã đăng nhập), Quản trị viên và Đối tác bán hàng.



Hình 1. Sơ đồ Use Case theo vai trò

3.3 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu gồm 14 bảng, được thiết kế lại hoàn toàn thành các bảng quan hệ chuẩn hoá ở giai đoạn MDM (thay vì lưu chuỗi tự do cho danh mục/thương hiệu như phiên bản khởi tạo ban đầu).



Hình 2. Sơ đồ thực thể quan hệ (ERD)

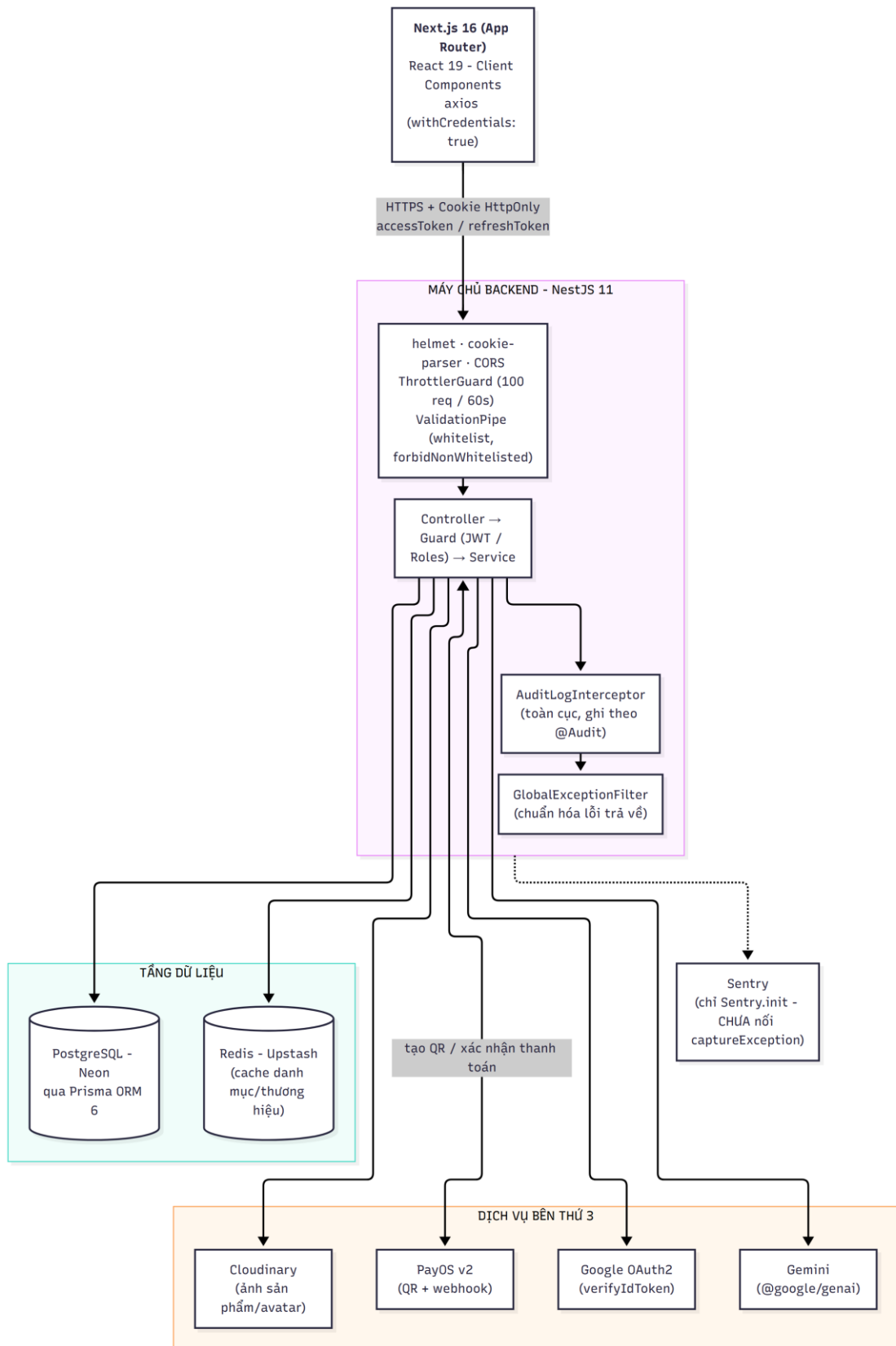
Bảng 1. Mô tả các thực thể chính trong cơ sở dữ liệu

Thực thể	Mô tả
User	Tài khoản người dùng (ADMIN/VENDOR/CUSTOMER). `ccd` và `dateOfBirth` là optional — không bắt buộc thu thập khi đăng ký.
RefreshToken	Token làm mới, lưu bản băm SHA-256 (không lưu token gốc), có thể `revoked` khi xoay vòng hoặc đăng xuất.
Category / SubCategory / Brand	Dữ liệu chủ (MDM) dạng bảng quan hệ thật, có FK, thay cho chuỗi tự do ở phiên bản đầu.
Product / ProductVariant	Sản phẩm và biến thể màu sắc; `slug` bắt buộc sau khi tạo; `specs` lưu dạng JSON linh hoạt.
Order / OrderItem	Đơn hàng snapshot toàn bộ thông tin tại thời điểm mua (địa chỉ, tên màu biến thể, giá) — không phụ thuộc dữ liệu gốc bị thay đổi sau này.
PaymentTransaction	Lưu lại từng lần webhook thanh toán xử lý, dùng để chống xử lý trùng (idempotency) theo `transactionId`.

Thực thể	Mô tả
Discount	Mã giảm giá, `percentage` lưu dạng thập phân (0.1 = 10%), không có quan hệ khoá ngoại tới Order (chỉ lưu snapshot mã).
Review	Đánh giá sản phẩm, ràng buộc `@@unique([userId, productId])` — 1 người chỉ đánh giá 1 sản phẩm 1 lần.
Address	Số địa chỉ giao hàng cá nhân, không liên kết khoá ngoại tới Order (Order tự snapshot địa chỉ riêng).
AuditLog	Nhật ký thao tác nhạy cảm, `userId` nullable với `onDelete: SetNull` — vẫn giữ log nếu tài khoản bị xoá.

3.4 Kiến trúc hệ thống

Hệ thống theo kiến trúc client-server tách biệt hoàn toàn: frontend Next.js chỉ giao tiếp với backend qua REST API (axios, gửi kèm cookie), backend NestJS đóng vai trò trung gian duy nhất giữa client và mọi dịch vụ dữ liệu/bên thứ ba — không có dịch vụ bên ngoài nào được gọi trực tiếp từ trình duyệt.

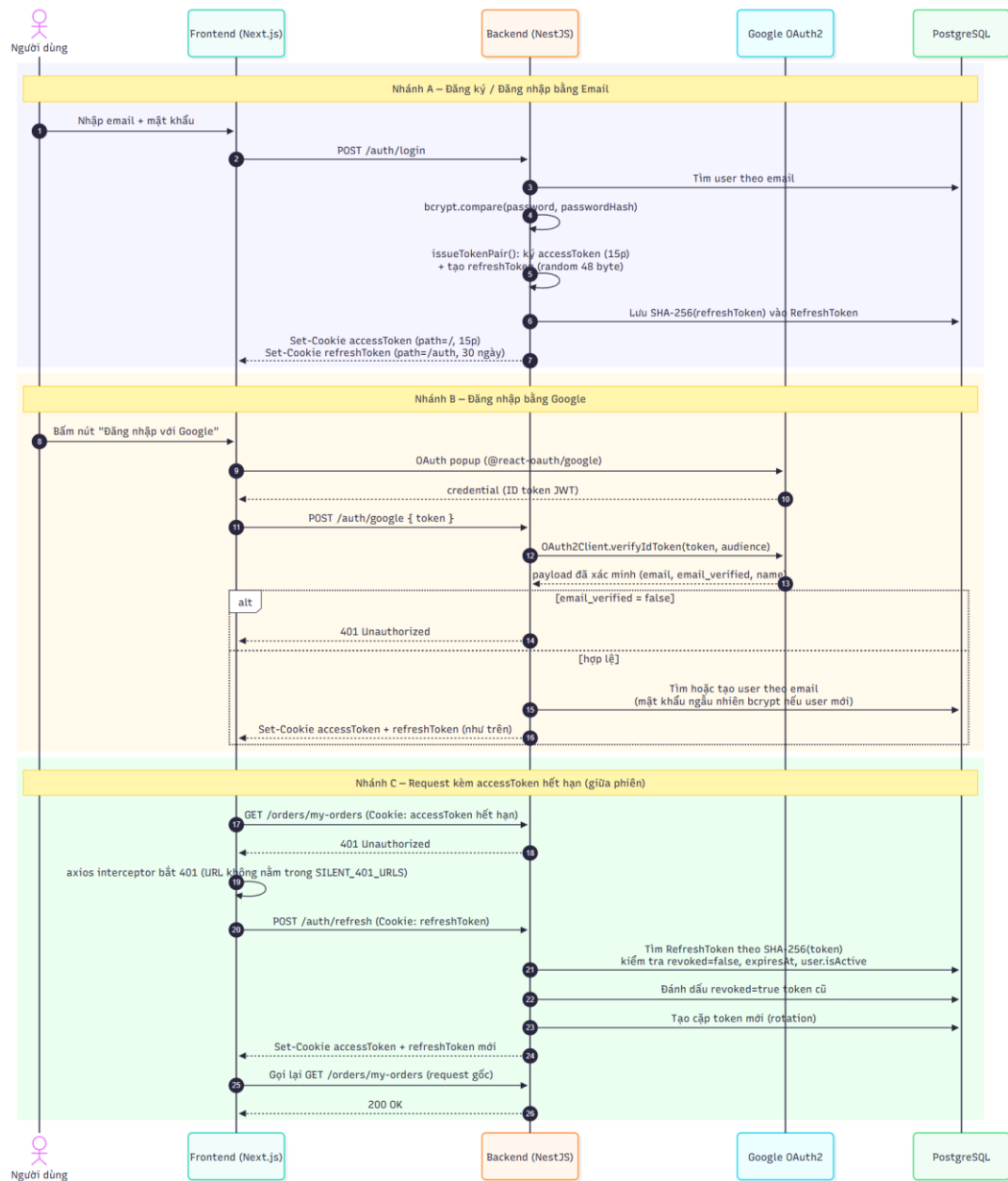


Hình 3. Kiến trúc hệ thống tổng quan

3.5 Các luồng nghiệp vụ quan trọng

3.5.1 Đăng ký / Đăng nhập và cơ chế Access — Refresh Token

Hệ thống hỗ trợ song song 2 luồng đăng nhập: bằng email/mật khẩu và bằng tài khoản Google. Điểm thiết kế quan trọng là accessToken có vòng đời ngắn (15 phút) nhằm giảm thiệt hại nếu bị đánh cắp, trong khi trải nghiệm người dùng vẫn liên tục nhờ cơ chế refresh tự động chạy ngầm ở axios interceptor — người dùng không hề nhận ra token vừa được xoay vòng.



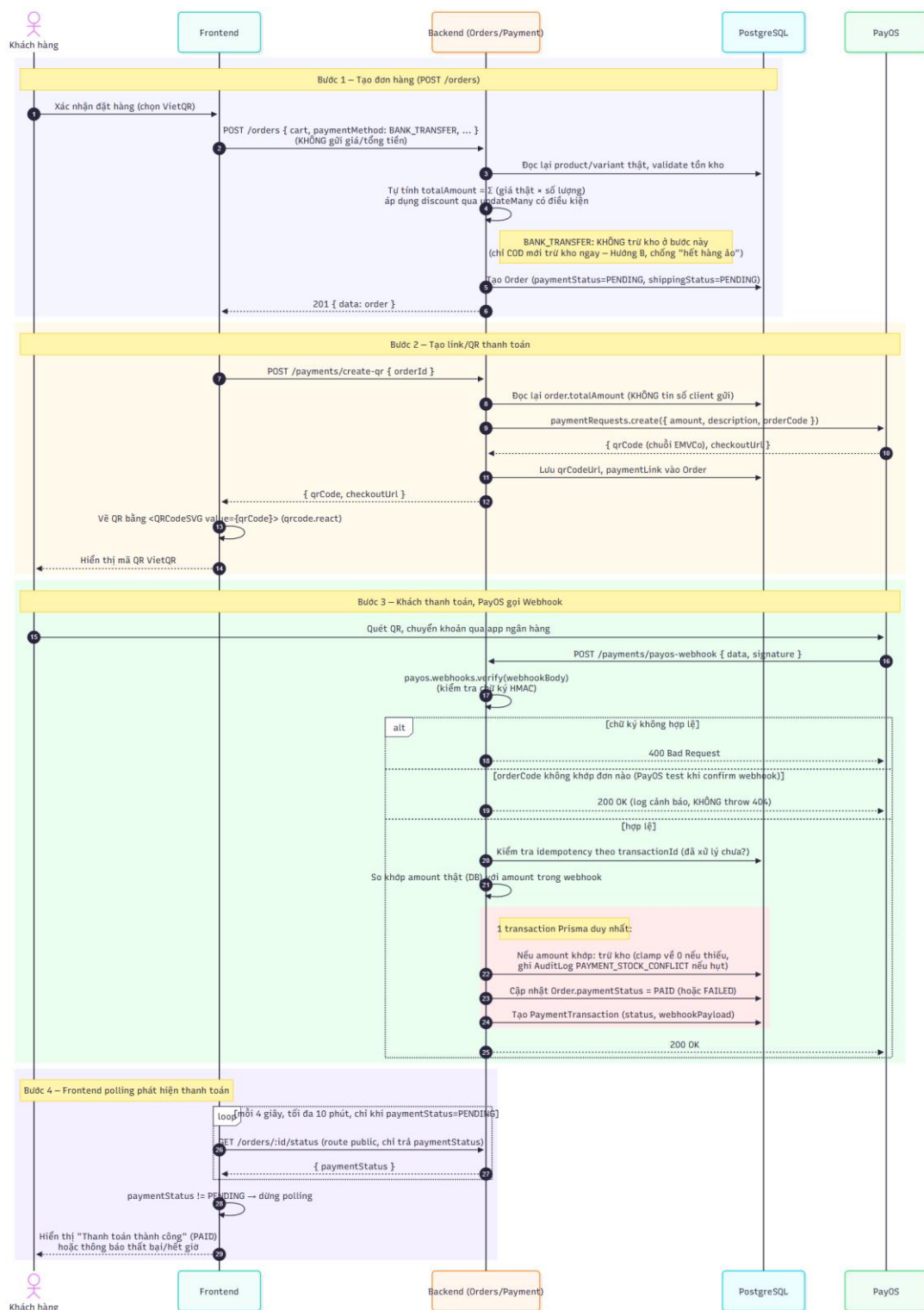
Hình 4. Luồng đăng ký/dăng nhập và cơ chế Access/Refresh Token

3.5.2 Đặt hàng và thanh toán VietQR — điểm nhấn kỹ thuật của đồ án

Đây là luồng nghiệp vụ phức tạp nhất và có giá trị kỹ thuật cao nhất trong đồ án, vì phải xử lý đúng một giao dịch **BẤT ĐỒNG BỘ**: thời điểm khách bấm "Đặt hàng" và thời

điểm PayOS xác nhận tiền đã về là 2 thời điểm hoàn toàn tách rời nhau, có thể cách nhau vài giây tới không bao giờ xảy ra (khách bỏ ngang không quét mã).

Câu hỏi thiết kế cốt lõi: nên trừ tiền kho vào lúc nào? Phương án ban đầu (trừ kho ngay lúc tạo đơn, giống hệ COD) gây ra hệ quả thực tế: mỗi đơn VietQR bị bỏ ngang không thanh toán sẽ giữ tiền kho "chết" vô thời hạn — nhiều khách hàng đặt thử rồi bỏ ngang có thể khiến sản phẩm hiển thị hết hàng dù không ai thực sự mua. Hệ thống áp dụng quyết định thiết kế: đơn COD trừ kho ngay (đặt đơn COD được xem là cam kết mua), còn đơn VietQR CHỈ trừ kho tại đúng thời điểm webhook xác nhận thanh toán thành công thật — chấp nhận đánh đổi là tiền kho không được "giữ chỗ" trong lúc khách đang thao tác quét mã, đổi lại tránh được vấn đề tiền kho ảo nghiêm trọng hơn nhiều về lâu dài.



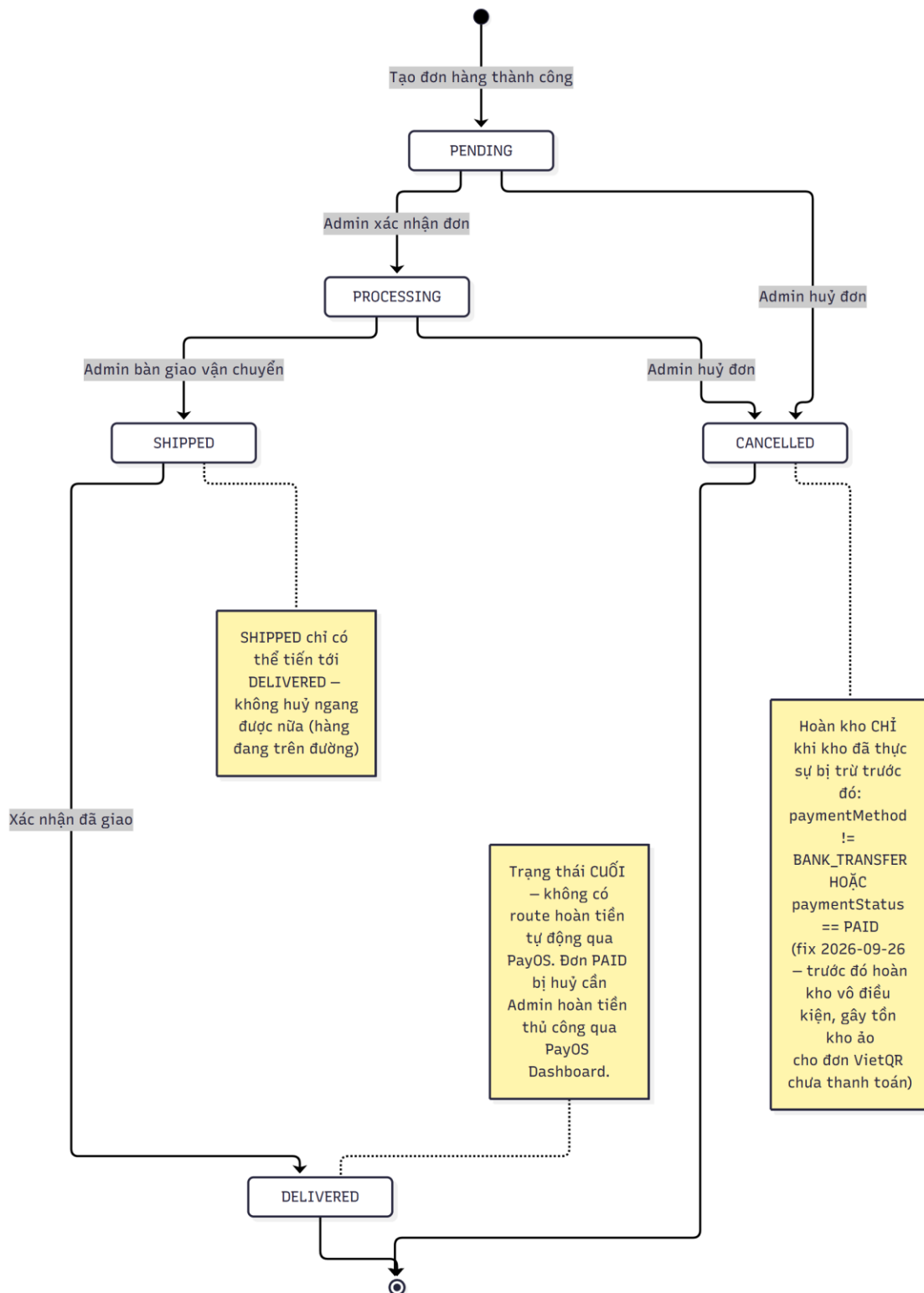
Hình 5. Luồng đặt hàng và thanh toán VietQR qua PayOS

Ba nguyên tắc bảo mật/toàn vẹn dữ liệu được áp dụng xuyên suốt luồng này: (1) server luôn tự tính lại toàn bộ số tiền từ giá thật trong cơ sở dữ liệu, không bao giờ tin số tiền do client gửi lên; (2) chữ ký webhook được xác minh bằng `payos.webhooks.verify()` trước khi xử lý bất kỳ dữ liệu nào bên trong; (3) toàn bộ thao tác trừ kho, cập nhật trạng

thái thanh toán và ghi nhận giao dịch nằm trong đúng 1 transaction cơ sở dữ liệu duy nhất — không thể xảy ra trạng thái nửa vời (VD: đã đánh dấu PAID nhưng chưa trừ kho).

3.5.3 Vòng đời trạng thái đơn hàng

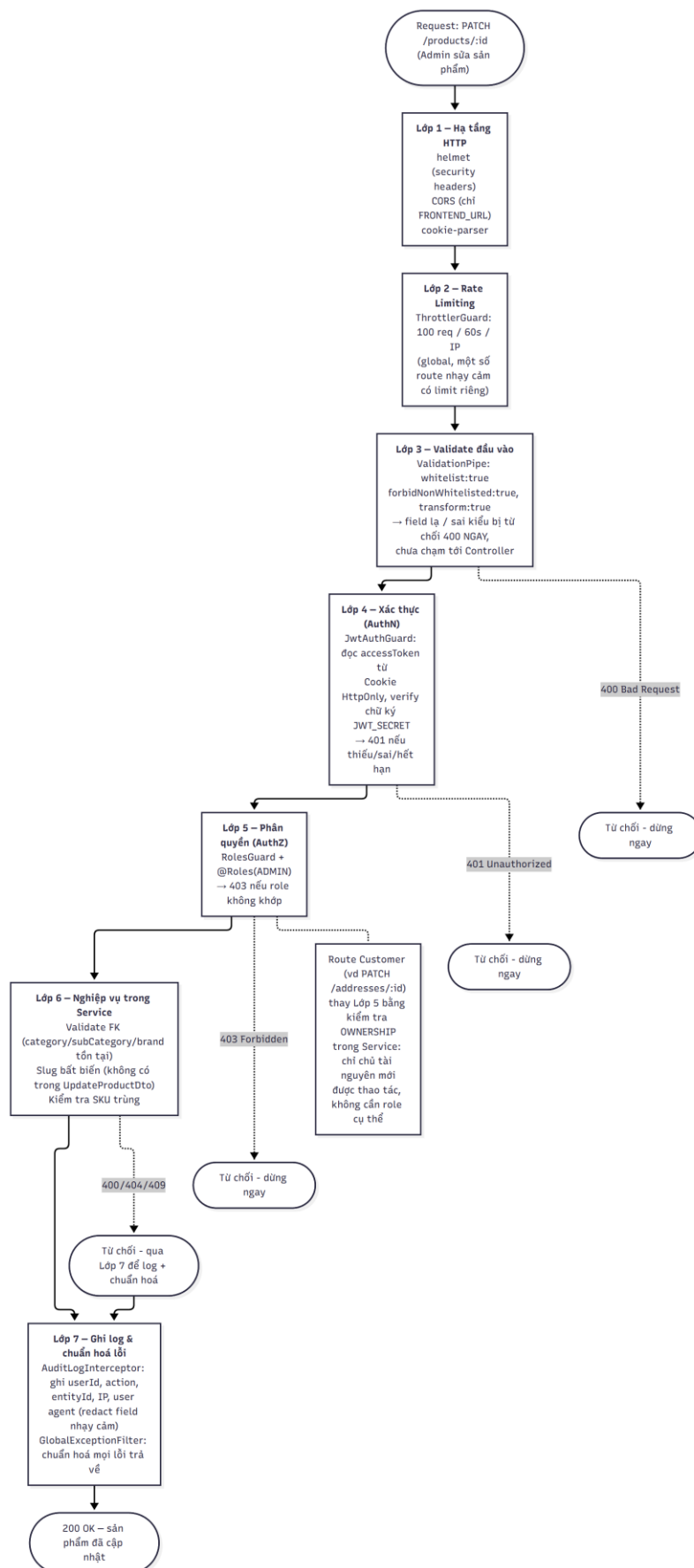
Trạng thái vận chuyển của đơn hàng (`shippingStatus`) được kiểm soát chặt bằng một state machine tường minh ở tầng service, không cho phép chuyển trạng thái tùy ý — ví dụ không thể chuyển thẳng từ PENDING sang DELIVERED, hay tác động lên một đơn đã ở trạng thái cuối (DELIVERED/CANCELLED).



Hình 6. Sơ đồ trạng thái vòng đời đơn hàng

3.5.4 Luồng bảo mật nhiều lớp

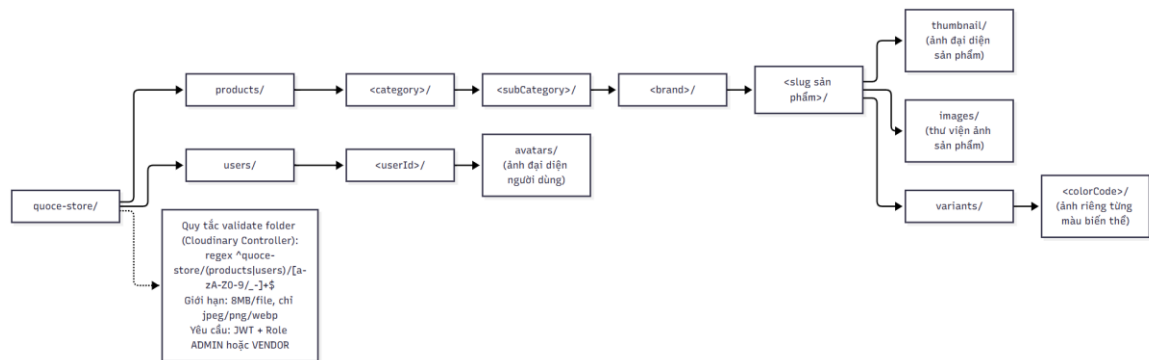
Mọi request tới các route nhạy cảm đều phải đi qua nhiều lớp kiểm tra độc lập trước khi chạm tới logic nghiệp vụ thật — nếu bất kỳ lớp nào từ chối, request dừng lại ngay, không lãng phí tài nguyên xử lý thêm.



Hình 7. Luồng bảo mật nhiều lớp minh hoạ qua PATCH /products/:id

3.5.5 Cấu trúc thư mục lưu trữ ảnh trên Cloudinary

Ảnh sản phẩm được tổ chức theo cây thư mục phân tầng phản ánh đúng cấu trúc dữ liệu (danh mục → danh mục con → thương hiệu → sản phẩm), giúp dễ tra cứu/dọn dẹp thủ công trên Cloudinary Dashboard khi cần, đồng thời được validate bằng regex ở tầng backend để chống Path Traversal khi client tự truyền tên thư mục.



Hình 8. Cấu trúc thư mục lưu ảnh trên Cloudinary

CHƯƠNG 4 — XÂY DỰNG & TRIỂN KHAI

4.1 Môi trường phát triển

- Hệ điều hành: Windows, PowerShell làm shell chính.
- Backend: Node.js, NestJS 11, Prisma 6, chạy dev bằng ``npm run start:dev`` (cổng 5000).
- Frontend: Next.js 16, React 19, chạy dev bằng ``npm run dev`` (cổng 3000).
- Cơ sở dữ liệu: PostgreSQL qua Neon (serverless, tự "ngủ" khi không hoạt động).
- Cache: Redis qua Upstash (region Singapore).
- Quản lý mã nguồn: Git, repository riêng trên GitHub.
- Kiểm thử thanh toán thật: ngrok để expose backend cục bộ ra Internet, phục vụ đăng ký webhook thật với PayOS trong quá trình phát triển.

4.2 Chi tiết các module đã xây dựng

4.2.1 Auth

Đăng ký/đăng nhập bằng email (bcrypt băm mật khẩu) và Google OAuth2, cấp cặp accessToken/refreshToken qua cookie HttpOnly, xoay vòng refreshToken khi hết hạn accessToken.

4.2.2 Users

Quản lý hồ sơ cá nhân (họ tên, SĐT, CCCD tự nguyện, giới tính, ngày sinh, avatar), đổi mật khẩu — toàn bộ route đều kiểm tra chủ sở hữu (ownership) ngoài việc bắt buộc đăng nhập.

4.2.3 Product / MDM (Categories, SubCategories, Brands)

Quản lý sản phẩm với biến thể màu, thông số kỹ thuật (specs) và điểm nổi bật (highlights) dạng danh sách động; danh mục/danh mục con/thương hiệu là bảng quan hệ thật có validate khoá ngoại và chặn xoá khi còn sản phẩm tham chiếu.

4.2.4 Orders / Payment

Xử lý đặt hàng, xác thực tồn kho, tích hợp thanh toán VietQR qua PayOS — chi tiết luồng đã trình bày ở mục 3.5.2.

4.2.5 Discounts

Mã giảm giá được validate và tính toán hoàn toàn ở server, chống race condition khi nhiều đơn cùng tranh lượt sử dụng cuối cùng của một mã.

4.2.6 AuditLog

Một interceptor toàn cục tự động ghi lại các thao tác nhạy cảm (được đánh dấu bằng decorator '@Audit') — bao gồm người thực hiện, hành động, đối tượng bị tác động, địa chỉ IP và User-Agent, có ẩn (redact) các trường nhạy cảm trong dữ liệu ghi log.

4.2.7 Reviews

Chỉ khách hàng có đơn hàng ở trạng thái DELIVERED cho đúng sản phẩm mới được đánh giá; điểm trung bình được tính động (on-the-fly) tại thời điểm truy vấn, không lưu trường tổng hợp trên bảng Product.

4.2.8 Addresses

Sổ địa chỉ cá nhân, tự động đề bạt địa chỉ khác làm mặc định khi địa chỉ mặc định bị xoá.

4.2.9 Cloudinary / AI

Upload ảnh yêu cầu xác thực và giới hạn định dạng/dung lượng; module AI dùng Gemini để tư vấn sản phẩm dựa trên danh sách sản phẩm đang bán, giới hạn 10 request/phút để kiểm soát chi phí gọi API.

4.2.10 Health

Endpoint kiểm tra sức khoẻ hệ thống bằng cách thực thi trực tiếp một truy vấn 'SELECT 1' tới cơ sở dữ liệu, loại trừ khỏi cơ chế giới hạn tần suất request.

4.3 Một số đoạn code tiêu biểu

Phần này trích nguyên văn 5 đoạn code có giá trị kỹ thuật cao nhất trong hệ thống, kèm giải thích ngắn gọn.

4.3.1 Đọc JWT từ Cookie HttpOnly (JwtStrategy)

Điểm mấu chốt: token được đọc trực tiếp từ cookie thay vì header Authorization, phù hợp với quyết định thiết kế không lưu token ở localStorage.

```
// backend/src/auth/strategies/jwt.strategy.ts
super({
  jwtFromRequest: (req: Request) => {
    return req?.cookies?.accessToken || null;
  },
  ignoreExpiration: false,
  secretOrKey: config.get<string>('JWT_SECRET')!,
});

async validate(payload: { sub: string; email: string; role: string }) {
  const user = await this.prisma.user.findUnique({
    where: { id: payload.sub },
  });
  if (!user || !user.isActive) {
    throw new UnauthorizedException(
      'Tài khoản không tồn tại hoặc đã bị vô hiệu hóa.'
    );
  }
  return { id: user.id, email: user.email, role: user.role };
}
```

Nguồn: *backend/src/auth/strategies/jwt.strategy.ts*

4.3.2 Chống race condition khi áp dụng mã giảm giá

Giữa lúc kiểm tra mã còn lượt dùng và lúc thực sự tăng bộ đếm, có thể có một đơn hàng khác cùng dùng mã này chen vào. `updateMany` với điều kiện `usedCount < maxUsage` ngay trong câu lệnh UPDATE đảm bảo chỉ một trong các giao dịch song song có thể thành công — nếu `count === 0` nghĩa là mã vừa hết lượt.

```
// backend/src/orders/orders.service.ts
const inc = await tx.discount.updateMany({
  where: {
    id: discount.id,
    ...(discount.maxUsage !== null && {
      usedCount: { lt: discount.maxUsage },
    }),
  },
  data: { usedCount: { increment: 1 } },
});
if (inc.count === 0) {
  throw new BadRequestException(
    'Mã giảm giá vừa hết lượt sử dụng, vui lòng bỏ mã và thử lại.'
  );
}
```

Nguồn: *backend/src/orders/orders.service.ts*

4.3.3 Xử lý Webhook thanh toán PayOS

Webhook được xác minh chữ ký trước khi xử lý bất kỳ dữ liệu nào; một trường hợp đặc biệt cần xử lý đúng là request kiểm tra tự động của PayOS khi đăng ký webhook (`webhooks.confirm()`) gửi kèm `orderCode` giả không tồn tại trong hệ thống — phải trả về HTTP 200 (không phải 404) để PayOS chấp nhận đăng ký URL.

```
// backend/src/payment/payment.service.ts
verifiedData = await this.payos.webhooks.verify(webhookBody);
...
const existingTransaction = await this.prisma.paymentTransaction
  .findUnique({ where: { transactionId } });
if (existingTransaction) {
  return { success: true, message: 'Giao dịch đã được xử lý trước đó' };
}

const order = await this.prisma.order.findUnique({ where: { orderCode } });
if (!order) {
  // PayOS tự gửi 1 request test (orderCode giả) khi confirm() webhook -
  // PHẢI trả 200, không phải 404, nếu không PayOS từ chối đăng ký URL.
  this.logger.warn(`Webhook orderCode ${orderCode} không khớp đơn nào`);
  return { success: true, message: 'Đã nhận webhook, không khớp đơn hàng.' };
}

const shouldDeductStock =
  isSuccess &&
  order.paymentMethod === 'BANK_TRANSFER' &&
  order.paymentStatus !== PaymentStatus.PAID;

await this.prisma.$transaction(async (tx) => {
  // trừ kho (clamp về 0 nếu thiếu) + cập nhật paymentStatus +
  // tạo PaymentTransaction - TẤT CẢ trong cùng 1 transaction
});
```

Nguồn: `backend/src/payment/payment.service.ts` (rút gọn, giữ nguyên phần lỗi)

4.3.4 Cập nhật biến thể sản phẩm: xóa sạch rồi tạo lại

Khi cập nhật sản phẩm có gửi kèm `variants`, hệ thống xóa toàn bộ biến thể cũ rồi tạo lại theo danh sách mới, thay vì so khớp từng bản ghi để cập nhật (upsert). Cách này an toàn vì `OrderItem.variantId` có `onDelete: SetNull` và tên màu đã được snapshot sẵn lúc mua hàng, nên đơn hàng cũ không bị ảnh hưởng dù ID biến thể thay đổi.

```
// backend/src/product/product.service.ts
if (dto.variants !== undefined) {
  await tx.productVariant.deleteMany({ where: { productId: id } });
}
const fallbackPrice = dto.price ?? product.price;

return await tx.product.update({
  where: { id },
  data: {
    ...data,
    ...(dto.variants !== undefined &&
```

```

    dto.variants.length > 0 && {
      variants: {
        create: dto.variants.map((v) => ({
          colorCode: v.colorCode,
          colorName: v.colorName,
          hexCode: v.hexCode,
          price: v.price && v.price > 0 ? v.price : fallbackPrice,
          stock: v.stock ?? 0,
          images: v.images ?? [],
        })),
      },
    },
  },
});

```

Nguồn: *backend/src/product/product.service.ts*

4.3.5 Chỉ hoàn kho khi kho đã thực sự bị trừ (fix ngày 26/09/2026)

Đây là một sửa lỗi thật được phát hiện trong quá trình rà soát phục vụ báo cáo này (xem chi tiết ở Chương 5). Hàm cập nhật trạng thái vận chuyển trước đây hoàn kho vô điều kiện khi đơn bị huỷ, không phân biệt phương thức thanh toán — trong khi đơn VietQR chưa thanh toán chưa từng bị trừ kho, dẫn tới tồn kho ảo cộng dồn sai mỗi khi huỷ một đơn VietQR còn đang chờ thanh toán.

```

// backend/src/orders/orders.service.ts
const stockWasDeducted =
  order.paymentMethod !== 'BANK_TRANSFER' ||
  order.paymentStatus === 'PAID';

if (dto.shippingStatus === 'CANCELLED' && stockWasDeducted) {
  for (const item of order.orderItems) {
    if (item.variantId) {
      await tx.productVariant.update({
        where: { id: item.variantId },
        data: { stock: { increment: item.quantity } },
      });
    } else {
      await tx.product.update({
        where: { id: item.productId },
        data: { stock: { increment: item.quantity } },
      });
    }
  }
}
}

```

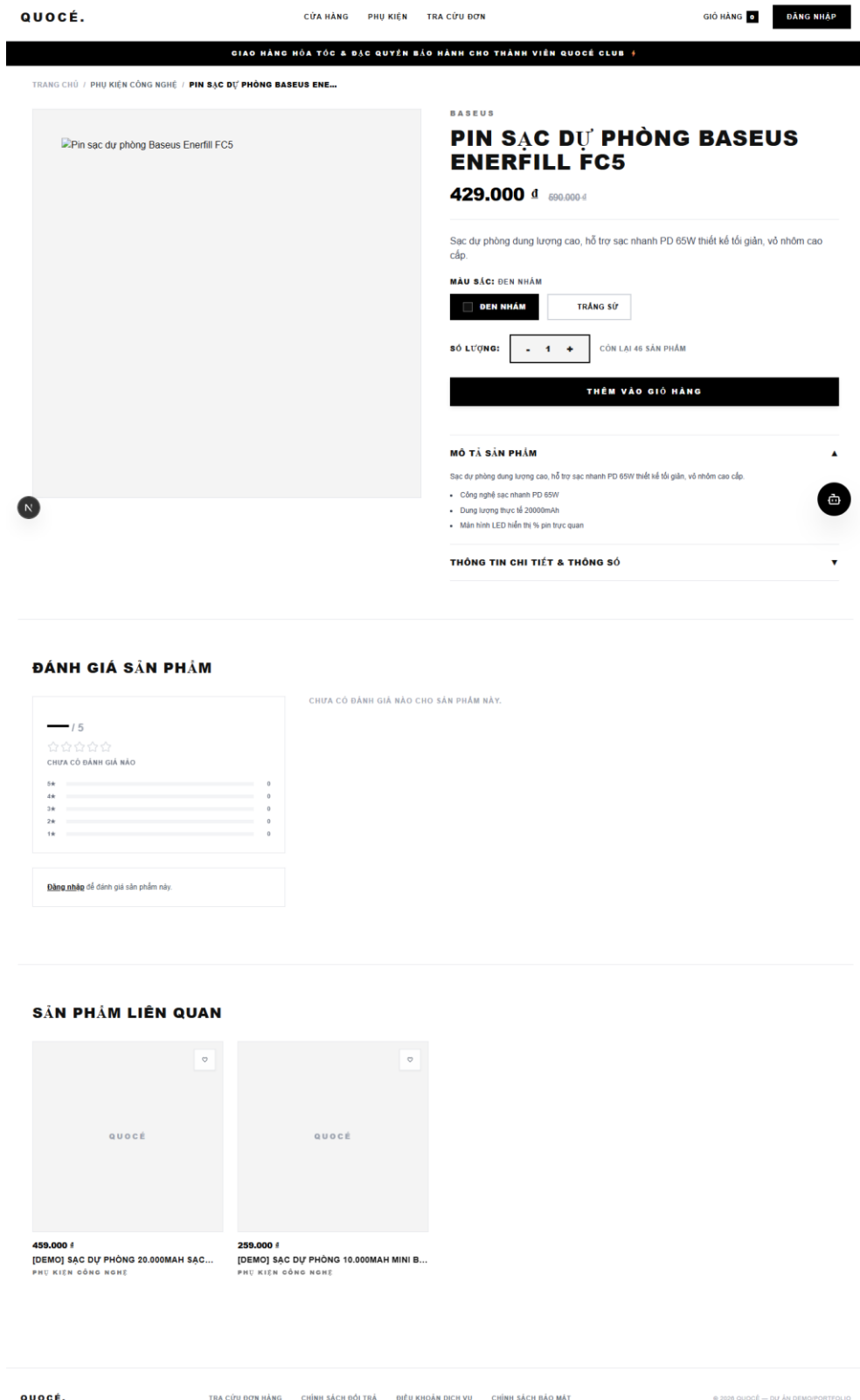
Nguồn: *backend/src/orders/orders.service.ts*

4.4 Giao diện chương trình

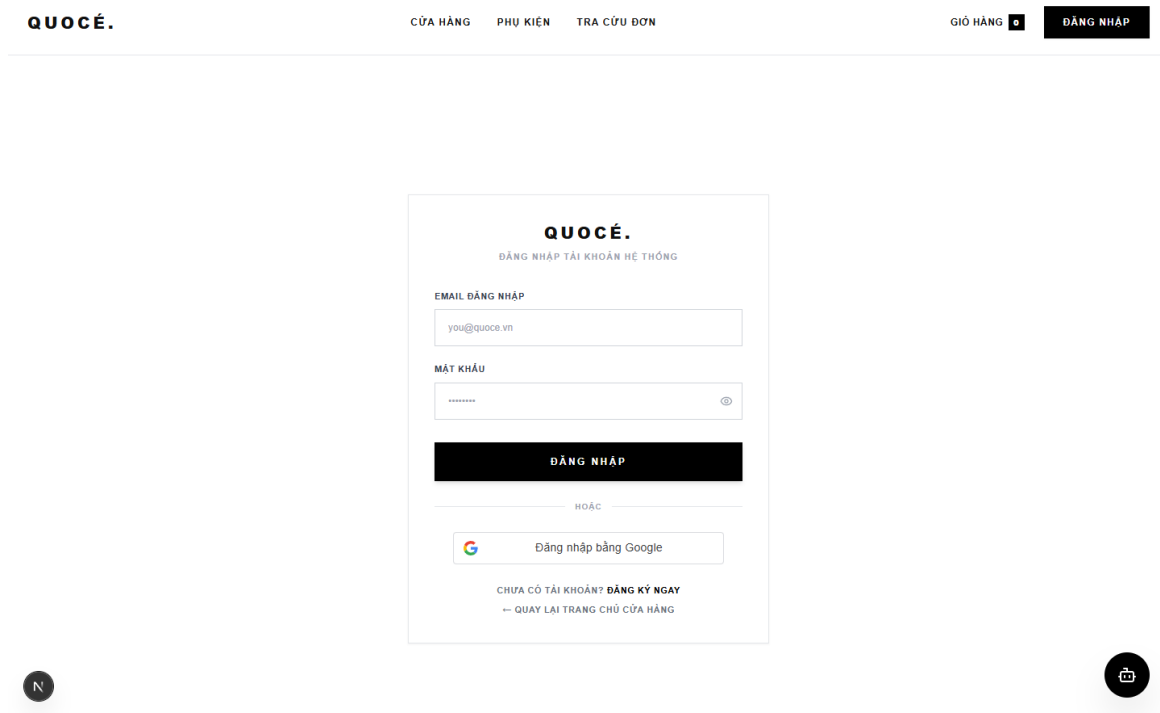
4.4.1 Storefront — Khách hàng



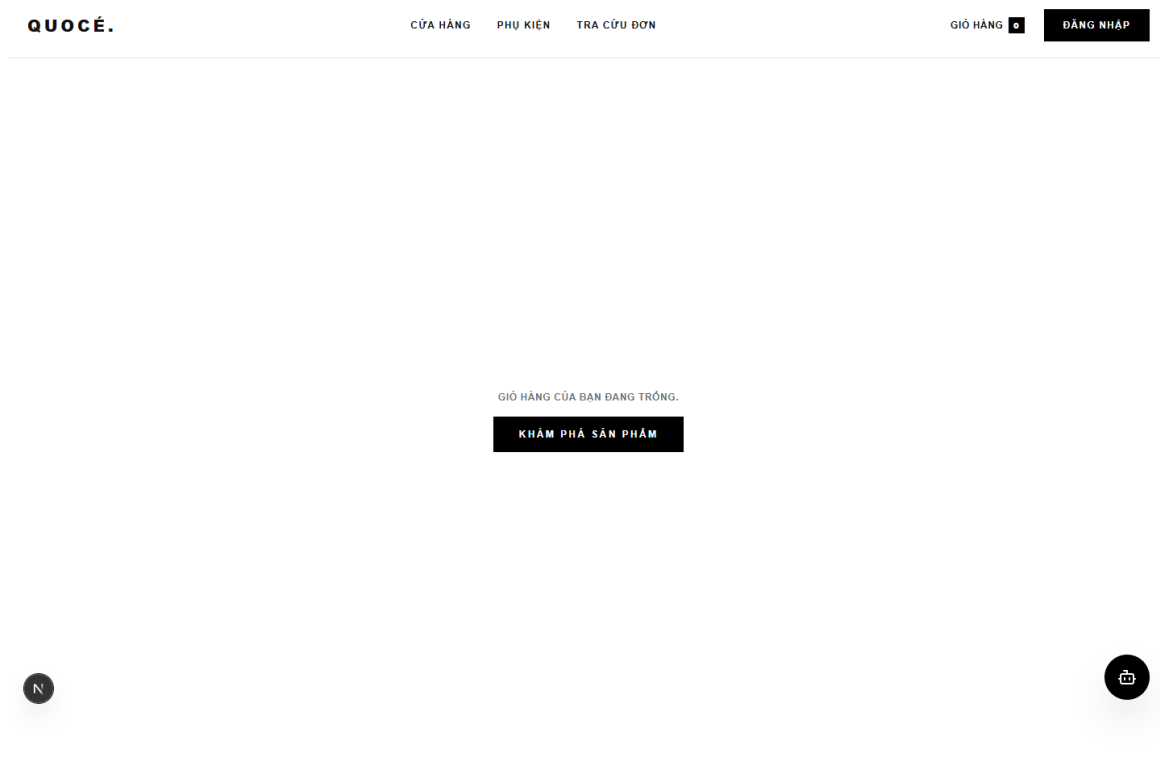
Hình 9. Trang chủ storefront



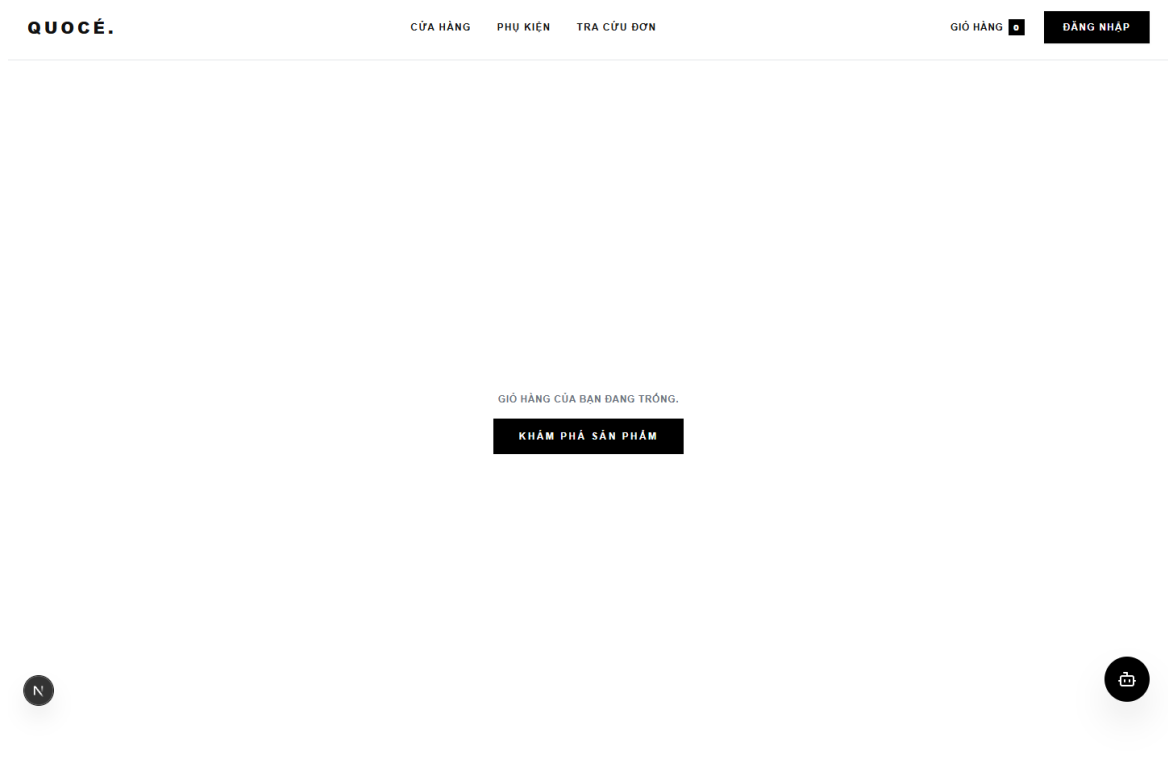
Hình 10. Trang chi tiết sản phẩm (chọn biến thể màu, đánh giá)



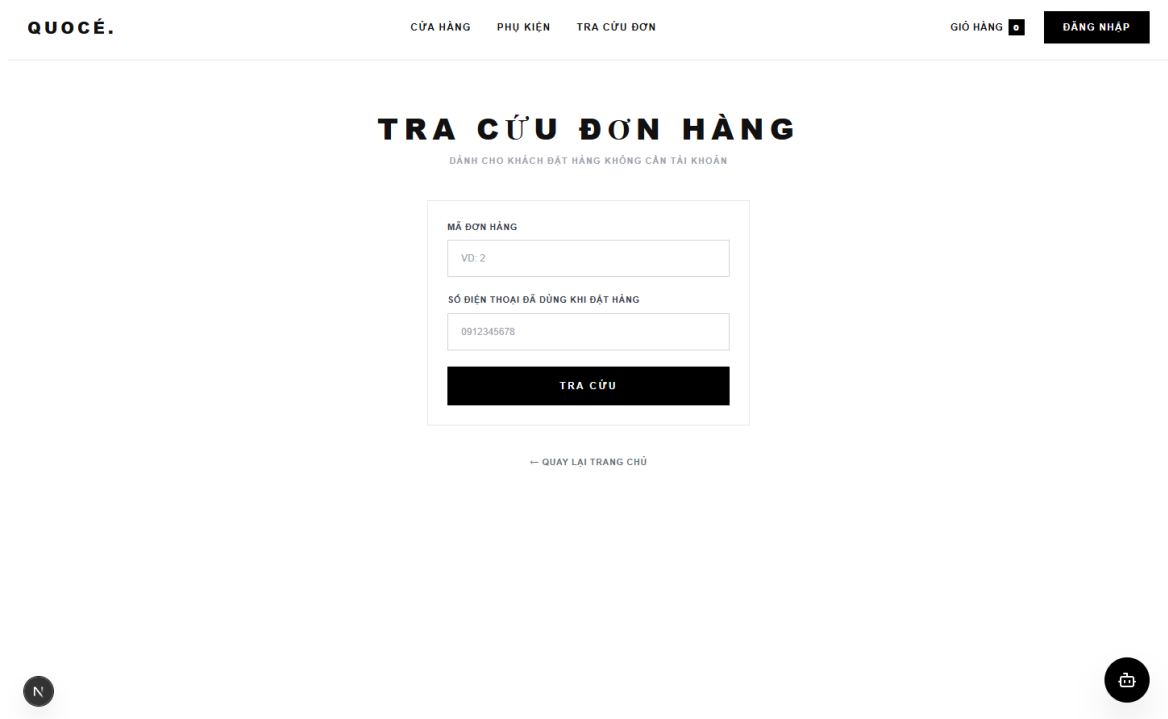
Hình 11. Trang đăng nhập (email và Google OAuth2)



Hình 12. Giỏ hàng — chọn từng dòng để thanh toán một phần



Hình 13. Trang thanh toán — chọn phương thức COD/VietQR, progress-step



Hình 14. Tra cứu đơn hàng dành cho khách vắng lại

ĐIỀU KHOẢN DỊCH VỤ

CẬP NHẬT LẦN CUỐI: 03/09/2020

1. ĐỊNH NGHĨA VÀ PHẠM VI ÁP DỤNG

Điều khoản này quy định quyền và nghĩa vụ giữa QUOCẾ ("chúng tôi") và người sử dụng website ("khách hàng", "bạn") khi truy cập, đăng ký tài khoản, hoặc đặt hàng qua hệ thống. Bằng việc sử dụng dịch vụ, bạn được xem là đã đọc và đồng ý với các điều khoản dưới đây.

2. ĐIỀU KIỆN SỬ DỤNG VÀ TÀI KHOẢN

- Bạn cam kết cung cấp thông tin chính xác khi đăng ký tài khoản và đặt hàng (họ tên, số điện thoại, địa chỉ giao hàng).
- Bạn có trách nhiệm bảo mật thông tin đăng nhập của mình, mọi hoạt động thực hiện dưới tài khoản của bạn được xem là do bạn thực hiện, từ trường hợp chúng minh được tài khoản bị truy cập trái phép.
- Chúng tôi có quyền tạm ngừng hoặc từ chối phục vụ tài khoản có dấu hiệu gian lận, lạm dụng hệ thống, hoặc vi phạm điều khoản này.
- Bạn có thể đặt hàng với vai trò khách vắng tại (không cần đăng ký tài khoản); một số quyền lợi (theo dõi lịch sử đơn hàng đầy đủ, tích điểm) chỉ áp dụng cho tài khoản đã đăng ký.

3. ĐẶT HÀNG VÀ GIÁ BÁN

Giá sản phẩm hiển thị trên website là giá cuối cùng do hệ thống xác định tại thời điểm bạn đặt hàng. Để bảo đảm tính minh bạch, tổng giá trị đơn hàng (bao gồm giảm giá nếu áp dụng mã hợp lệ) luôn được hệ thống tự tính toán tại từ dữ liệu gốc lưu trên máy chủ tại thời điểm xử lý đơn hàng — không dựa trên số liệu do tính duyệt của khách hàng gửi lên, nhằm ngăn ngừa sai lệch giá do lỗi kỹ thuật hoặc can thiệp trái phép.

Đơn hàng được xác nhận là hợp lệ sau khi hệ thống kiểm tra đủ tồn kho cho từng sản phẩm trong đơn. Với đơn thanh toán khi nhận hàng (COD), tồn kho được giữ ngay khi đơn được tạo. Với đơn thanh toán chuyển khoản (VietQR), tồn kho chỉ chính thức được trừ khi hệ thống xác nhận đã nhận được thanh toán — nếu bạn không hoàn tất thanh toán, đơn sẽ không được xử lý tiếp và tồn kho sản phẩm không bị ảnh hưởng.

4. HỦY ĐƠN HÀNG

Đơn hàng có thể được hủy trong các trường hợp và điều kiện sau:

- Đơn đang ở trạng thái **Chờ xử lý** hoặc **Đang xử lý** có thể được hủy.
- Đơn đã chuyển sang trạng thái **Đã giao vận** hoặc **Đã giao hàng** không thể hủy qua hệ thống — trường hợp này, vui lòng liên hệ trực tiếp để được hỗ trợ theo quy trình đổi trả (xem [Chính sách đổi trả](#)).
- Khi một đơn đã thanh toán bị hủy, việc hoàn tiền được xử lý theo đúng quy định tại [Chính sách đổi trả và hoàn tiền](#).

5. PHƯƠNG THỨC THANH TOÁN

- Thanh toán khi nhận hàng (COD):** bạn thanh toán trực tiếp cho đơn vị vận chuyển khi nhận hàng.
- Chuyển khoản VietQR:** thanh toán qua cổng trung gian PayOS. Đơn hàng chỉ được xác nhận thanh toán thành công sau khi hệ thống nhận được xác thực giao dịch từ PayOS — quá trình này thường diễn ra trong vài giây sau khi bạn hoàn tất chuyển khoản.

6. VẬN CHUYỂN

Thời gian giao hàng có thể thay đổi tùy khu vực địa lý và đơn vị vận chuyển được sử dụng. Trạng thái giao hàng của từng đơn (Chờ xử lý / Đang xử lý / Đã giao vận / Đã giao hàng) được cập nhật và hiển thị minh bạch tại trang lịch sử đơn hàng của bạn.

7. QUYỀN VÀ NGHĨA VỤ CỦA QUOCẾ

- Cung cấp thông tin sản phẩm trung thực, xử lý đơn hàng đúng theo trạng thái đã cam kết.
- Bảo vệ dữ liệu cá nhân của khách hàng theo đúng [Chính sách bảo mật](#).
- Có quyền từ chối xử lý đơn hàng có dấu hiệu gian lận hoặc thông tin giao nhận không xác thực được.
- Có quyền thay đổi, cập nhật điều khoản này; phiên bản áp dụng là phiên bản được đăng công khai tại thời điểm bạn đặt hàng.

8. QUYỀN VÀ NGHĨA VỤ CỦA KHÁCH HÀNG

- Cung cấp thông tin giao nhận chính xác; chịu trách nhiệm nếu đơn hàng không giao được do thông tin sai.
- Được quyền tra cứu trạng thái đơn hàng bất kỳ lúc nào qua tài khoản hoặc chức năng tra cứu dành cho khách vắng tại.
- Được quyền yêu cầu đổi trả/hoàn tiền theo đúng điều kiện tại [Chính sách đổi trả](#).

9. GIỚI HẠN TRÁCH NHIỆM

Chúng tôi không chịu trách nhiệm đối với các thiệt hại phát sinh do nguyên nhân bất khả kháng (thiên tai, gián đoạn hạ tầng internet/bến thư ba ngoài khả năng kiểm soát hợp lý), hoặc do khách hàng cung cấp thông tin giao nhận không chính xác. Trách nhiệm của chúng tôi đối với mỗi đơn hàng không vượt quá giá trị thực tế của đơn hàng đó.

10. SỞ HỮU TRI TUỆ

Tất cả nội dung, hình ảnh, thiết kế giao diện, và mã nguồn hiển thị trên website thuộc quyền sở hữu của QUOCẾ hoặc được cấp phép sử dụng hợp pháp. Việc khi có ghi chú khác: Nghiêm cấm sao chép, phân phối lại vì mục đích thương mại mà không có sự cho phép bằng văn bản.

11. GIẢI QUYẾT TRANH CHẤP

Mọi tranh chấp phát sinh trước tiên sẽ được giải quyết thông qua thương lượng, hòa giải trên cơ sở thiện chí giữa hai bên. Trường hợp không đạt được thỏa thuận, tranh chấp sẽ được giải quyết tại cơ quan có thẩm quyền theo quy định của pháp luật Việt Nam.

12. LUẬT ÁP DỤNG VÀ THAY ĐỔI ĐIỀU KHOẢN

Điều khoản này được xây dựng và giải thích theo pháp luật Việt Nam. Chúng tôi có thể cập nhật điều khoản theo thời gian; phiên bản mới nhất luôn được đăng công khai tại trang này.

Mọi thắc mắc liên quan tới điều khoản dịch vụ, vui lòng liên hệ: support@quocce.vn.

Đây là bản soạn thảo tham khảo dựa trên khung pháp lý hiện hành, được xây dựng để minh họa năng lực triển khai điều khoản dịch vụ cho một hệ thống thương mại điện tử. Khuyến nghị rà soát bởi chuyên gia pháp lý trước khi áp dụng cho hoạt động kinh doanh thực. Nội dung này KHÔNG PHẢI tư vấn pháp lý chính thức.

Hình 15. Trang Điều khoản dịch vụ

[CẦN NGƯỜI DÙNG TỰ CHÈN ẢNH CHỤP MÀN HÌNH: màn hình mã QR VietQR sau khi đặt hàng và màn hình xác nhận "Thanh toán thành công" — không

chụp tự động vì bước này cần tạo 1 đơn hàng thật và quét mã bằng ứng dụng ngân hàng thật]

4.4.2 Tài khoản khách hàng

QUOCẾ.

CỬA HÀNGPHỤ KIỆN

GIỎ HÀNG0

SYSTEM ADMINIS...

LỊCH SỬ ĐƠN HÀNG

MÃ ĐƠN: #33
5/9/2028

ĐÃ THANH TOÁNCHỜ XỬ LÝ

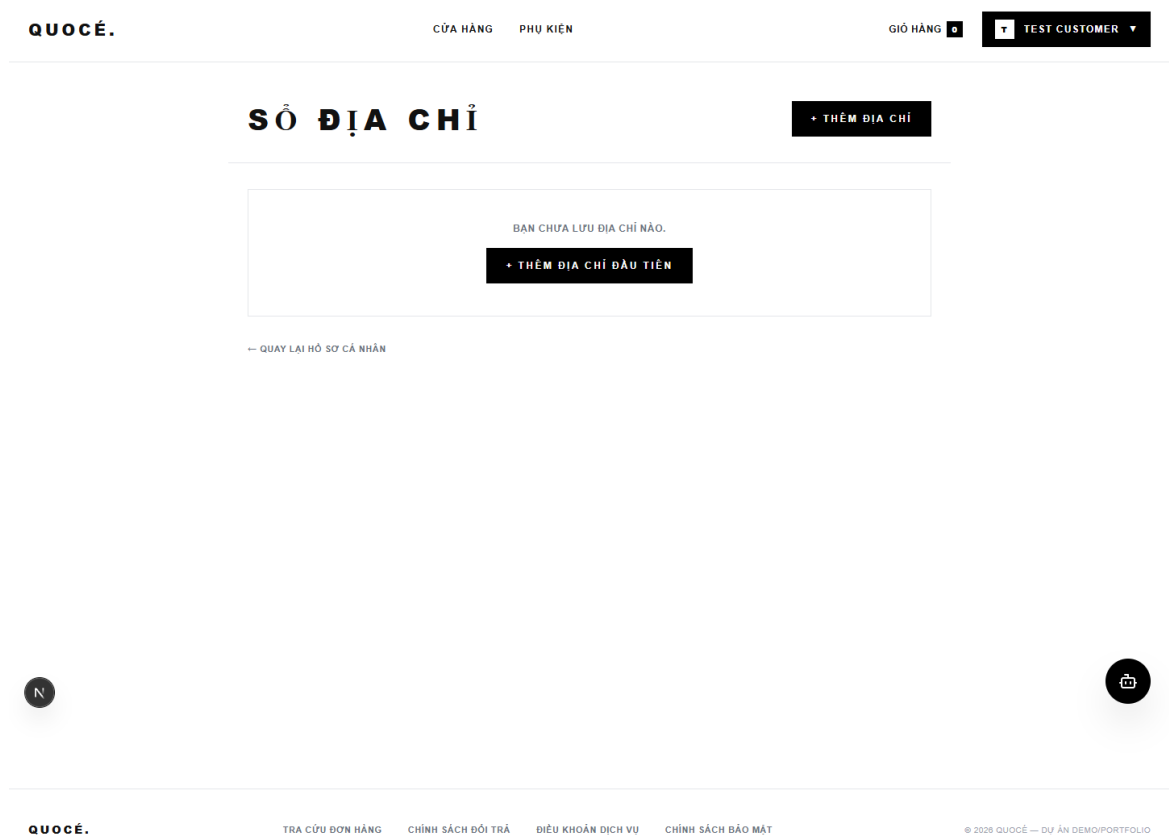
	TEST2 Màu: Đen nhám Số lượng: 1	1.000 đ
	TEST2 Màu: Trắng sứ Số lượng: 1	1.000 đ
XEM CHI TIẾT		TỔNG TIỀN: 2.000 đ

MÃ ĐƠN: #32
5/9/2028

ĐANG CHỜ THANH TOÁNCHỜ XỬ LÝ

	ABC Màu: Trắng sứ Số lượng: 3	369.270 đ
	ABC Màu: Xanh dương Số lượng: 3	369.270 đ

Hình 16. Lịch sử đơn hàng cá nhân



Hình 17. Sổ địa chỉ giao hàng

4.4.3 Trang quản trị (Admin)

QUOCÉ WORLD-CLASS ENTERPRISE PIM

QUẢN LÝ SẢN PHẨM & BIẾN THỂ

+ THÊM SẢN PHẨM MỚI

TÌM KIẾM THEO TÊN SẢN PHẨM HOẶC MÃ SKU...

TẤT CẢ DANH MỤC

TẤT CẢ DANH MỤC CON

TẤT CẢ THƯƠNG HIỆU

HÌNH ẢNH	MÃ SKU	TÊN SẢN PHẨM	DANH MỤC	THƯƠNG HIỆU	GIÁ BÁN	BIẾN THỂ MÀU	KHO	TRANG THÁI	THAO TÁC
	QUO-ANKER-TEST2-XS8A	TEST2	PHỤ KIỆN CÔNG NGHỆ	ANKER	1.000 đ		Tổng biến thể: 22	ĐANG BÁN	SỬA XÓA
	QUO-ANKER-BACCBAS-211M	BACCBAS	PHỤ KIỆN CÔNG NGHỆ	BASEUS	100.000 đ		Tổng biến thể: 95	ĐANG BÁN	SỬA XÓA
	QUO-ANKER-ABC-LSH2	ABC	PHỤ KIỆN CÔNG NGHỆ	BASEUS	123.090 đ		Tổng biến thể: 45	ĐANG BÁN	SỬA XÓA
	QUO-BASEUS-DEMOABC-UQA2	DEMOABC	PHỤ KIỆN CÔNG NGHỆ	SOUNDCORE	349.000 đ	Đơn lẻ	28	ĐANG BÁN	SỬA XÓA
	DEMO-CHUOTKHONGDA-1EQ9	[DEMO] CHUỘT KHÔNG D...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	199.000 đ	Đơn lẻ	44	ĐANG BÁN	SỬA XÓA
	DEMO-GIADOLAPTOP-2EXK	[DEMO] GIÁ ĐỠ LAPTOP ...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	299.000 đ	Đơn lẻ	25	ĐANG BÁN	SỬA XÓA
	DEMO-GIADODIENTHONG-VOY2	[DEMO] GIÁ ĐỠ ĐIỆN TH...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	79.000 đ	Đơn lẻ	90	ĐANG BÁN	SỬA XÓA
	DEMO-CUSACGANHSD-BOPC	[DEMO] CÚ SẠC GAN 65...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	399.000 đ	Đơn lẻ	30	ĐANG BÁN	SỬA XÓA
	DEMO-CUSACNHANHSD-ASB3	[DEMO] CÚ SẠC NHANH ...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	149.000 đ	Đơn lẻ	70	ĐANG BÁN	SỬA XÓA
	DEMO-LOTCHUOTDAPU-1T3L	[DEMO] LỘT CHUỘT DA P...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	179.000 đ	Đơn lẻ	40	ĐANG BÁN	SỬA XÓA

TRANG 1 / 3 — TỔNG 22 SẢN PHẨM

Trang trước

Trang sau

Hình 18. Trang quản lý sản phẩm — phân trang, tìm kiếm, lọc theo danh mục/thương hiệu

THÊM SẢN PHẨM MỚI (PIM)

X ĐÓNG

TÊN SẢN PHẨM

MÃ SKU TỰ ĐỘNG

tr Sinh tại mã khác

VD: SẠC DỤ PHÒNG BASEUS...

Sẽ tự sinh khi nhập Tên sản phẩm

ĐỊNH DANH URL (SLUG)

DANH MỤC CHÍNH

+ Thêm mới

CHỌN DANH MỤC CON

THƯƠNG HIỆU (BRAND)

+ Thêm mới

GIÁ BÁN (VND)

GIÁ GỐC (ORIGINAL PRICE)

TỔNG TỐN KHO

429000

590000

50

ẢNH ĐẠI DIỆN CHÍNH (THUMBNAIL / CLOUDINARY UPLOAD)

TẢI ẢNH LÊN

BỘ SƯU TẬP ẢNH SẢN PHẨM (GALLERY IMAGES)

+ Thêm

MÔ TẢ CHI TIẾT (RICH CONTENT)

Nhập nội dung bài viết mô tả sản phẩm...

TRANG 1 / 3 — TỔNG 22 SẢN PHẨM

Trang trước

Trang sau

NO IMG	DEMO-GIADOOIENTHO- VOY2	[DEMO] GIÁ ĐỒ ĐIỆN TH...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	79.000 đ	Đơn lẻ	90	ĐANG BÁN	SỬA XÓA
NO IMG	DEMO-CUSACGANWSD- BOPC	[DEMO] CỬ SẠC GAN 65...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	399.000 đ	Đơn lẻ	30	ĐANG BÁN	SỬA XÓA
NO IMG	DEMO-CUSACNHANHPD- ASB3	[DEMO] CỬ SẠC NHANH ...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	149.000 đ	Đơn lẻ	70	ĐANG BÁN	SỬA XÓA
NO IMG	DEMO-LOTCHUOTDAPU- 1T3L	[DEMO] LỚT CHUỘT DA P...	PHỤ KIỆN CÔNG NGHỆ	BASEUS	179.000 đ	Đơn lẻ	40	ĐANG BÁN	SỬA XÓA

QUỐC.

TRA CỨU ĐƠN HÀNG

CHÍNH SÁCH ĐỔI TRẢ

ĐIỀU KHOẢN DỊCH VỤ

CHÍNH SÁCH BẢO MẬT

© 2028 QUỐC — DỰ ÁN DEMO/PORTFOLIO

Hình 19. Form thêm sản phẩm mới — SKU tự sinh, danh mục/thương hiệu chọn nhanh

QUOCÉ WORLD-CLASS ENTERPRISE PIM

QUẢN LÝ ĐƠN HÀNG

TÌM THEO MÃ ĐƠN / SĐT / TÊN KHÁCH...

TẤT CẢ TRẠNG THÁI

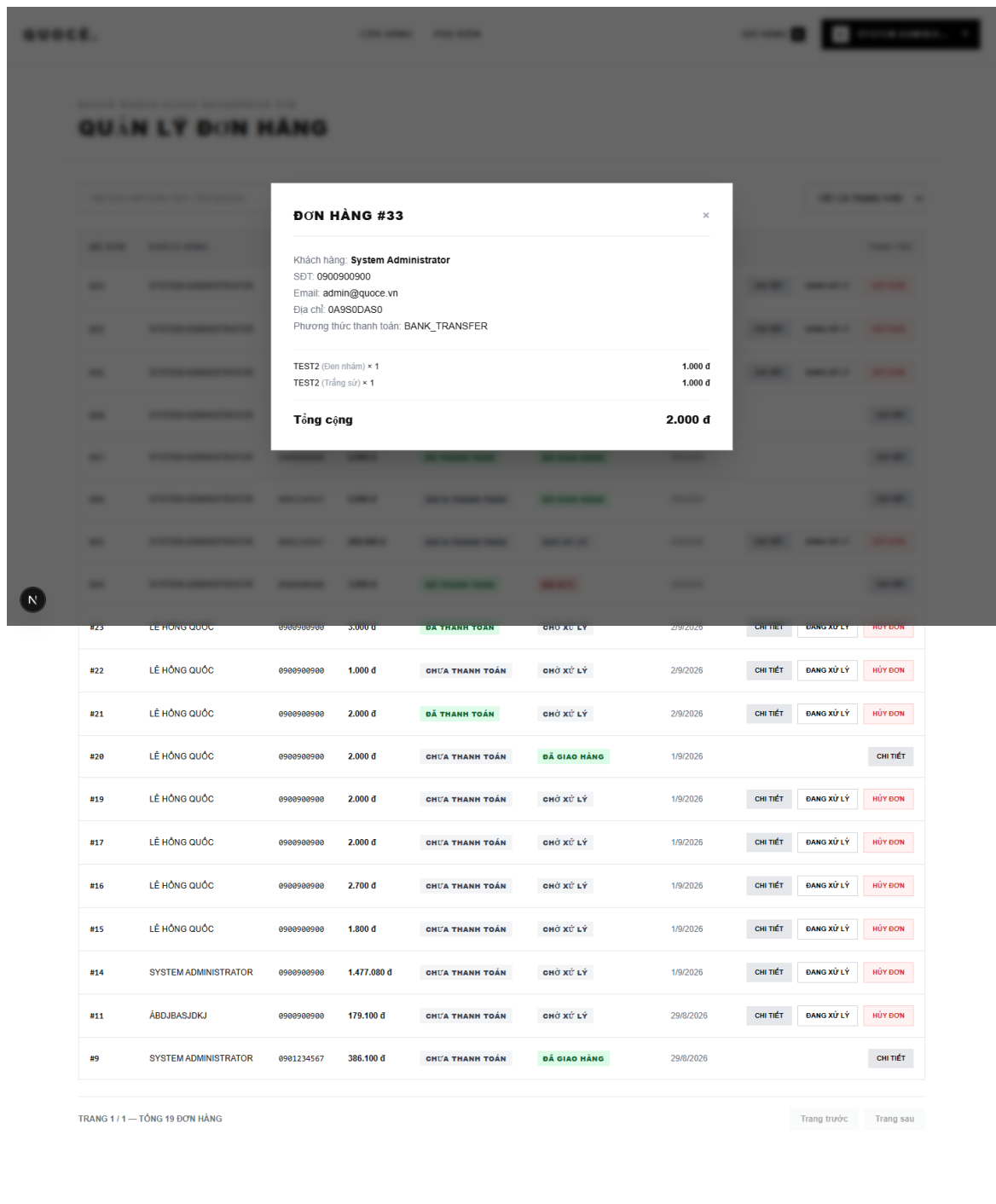
MÃ ĐƠN	KHÁCH HÀNG	SĐT	TỔNG TIỀN	THANH TOÁN	TRẠNG THÁI GIAO HÀNG	NGÀY TẠO	THAO TÁC		
#33	SYSTEM ADMINISTRATOR	0900900900	2.000 đ	ĐÃ THANH TOÁN	CHỜ XỬ LÝ	5/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#32	SYSTEM ADMINISTRATOR	0900900900	2.544.540 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	5/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#31	SYSTEM ADMINISTRATOR	0900900900	602.343 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	5/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#28	SYSTEM ADMINISTRATOR	0901234567	2.000 đ	ĐÃ THANH TOÁN	ĐÃ GIAO HÀNG	2/9/2026	CHI TIẾT		
#27	SYSTEM ADMINISTRATOR	0900900900	2.000 đ	ĐÃ THANH TOÁN	ĐÃ GIAO HÀNG	2/9/2026	CHI TIẾT		
#26	SYSTEM ADMINISTRATOR	0901234567	5.000 đ	CHƯA THANH TOÁN	ĐÃ GIAO HÀNG	2/9/2026	CHI TIẾT		
#25	SYSTEM ADMINISTRATOR	0901234567	200.000 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	2/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#24	SYSTEM ADMINISTRATOR	0900900900	3.000 đ	ĐÃ THANH TOÁN	ĐÃ HỦY	2/9/2026	CHI TIẾT		
#23	LÊ HỒNG QUỐC	0900900900	3.000 đ	ĐÃ THANH TOÁN	CHỜ XỬ LÝ	2/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#22	LÊ HỒNG QUỐC	0900900900	1.000 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	2/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#21	LÊ HỒNG QUỐC	0900900900	2.000 đ	ĐÃ THANH TOÁN	CHỜ XỬ LÝ	2/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#20	LÊ HỒNG QUỐC	0900900900	2.000 đ	CHƯA THANH TOÁN	ĐÃ GIAO HÀNG	1/9/2026	CHI TIẾT		
#19	LÊ HỒNG QUỐC	0900900900	2.000 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	1/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#17	LÊ HỒNG QUỐC	0900900900	2.000 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	1/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#16	LÊ HỒNG QUỐC	0900900900	2.700 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	1/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#15	LÊ HỒNG QUỐC	0900900900	1.800 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	1/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#14	SYSTEM ADMINISTRATOR	0900900900	1.477.080 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	1/9/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#11	ABDJBASJKJ	0900900900	179.100 đ	CHƯA THANH TOÁN	CHỜ XỬ LÝ	29/8/2026	CHI TIẾT	ĐANG XỬ LÝ	HỦY ĐƠN
#9	SYSTEM ADMINISTRATOR	0901234567	386.100 đ	CHƯA THANH TOÁN	ĐÃ GIAO HÀNG	29/8/2026	CHI TIẾT		

TRANG 1 / 1 — TỔNG 19 ĐƠN HÀNG

Trang trước

Trang sau

Hình 20. Trang quản lý đơn hàng — trạng thái theo state machine



Hình 21. Modal chi tiết đơn hàng và các nút chuyển trạng thái hợp lệ

CHƯƠNG 5 — KIỂM THỬ HỆ THỐNG

5.1 Kế hoạch kiểm thử

Do phạm vi đề án không xây dựng bộ unit test tự động (các file `\*.spec.ts` hiện tại là mã khung mặc định do NestJS CLI sinh ra, chưa được viết lại thành test nghiệp vụ thật — đây là hạn chế được nêu rõ ở Chương 6, không che giấu), việc kiểm thử được thực hiện bằng 4 phương pháp kết hợp:

- Gọi API thật bằng công cụ dòng lệnh (`Invoke-RestMethod`/PowerShell, `curl`) để xác nhận response đúng định dạng và mã trạng thái HTTP mong đợi.
- Viết script Node.js tự ký JSON Web Token bằng đúng `JWT\_SECRET` cấu hình thật, giả lập nhiều vai trò (ADMIN/CUSTOMER) để kiểm thử các route yêu cầu xác thực mà không cần thao tác đăng nhập thủ công lặp lại.
- Kiểm thử giao diện thủ công theo từng bước cụ thể đã liệt kê trước, đặc biệt với các luồng có trạng thái phức tạp (đăng nhập, thanh toán, quản trị đơn hàng).
- Kiểm thử thanh toán VietQR bằng giao dịch ngân hàng THẬT thông qua ngrok để xác nhận webhook PayOS hoạt động đúng trong điều kiện gần giống production.

Sau mỗi lần kiểm thử tạo dữ liệu tạm (đơn hàng, sản phẩm thử), dữ liệu này đều được dọn sạch khỏi cơ sở dữ liệu và xác nhận lại bằng truy vấn đếm số bản ghi trước khi ghi nhận hoàn thành.

5.2 Kết quả kiểm thử

Bảng 2. Tổng hợp một số kết quả kiểm thử tiêu biểu (số liệu thật, trích từ nhật ký phát triển)

Chức năng	Phương pháp	Kết quả cụ thể	Kết luận
Phân trang sản phẩm (backend)	Invoke-RestMethod (PowerShell)	page/limit/filter đúng, 400 với tham số sai (limit=999, categoryId=ALL...)	PASS
Mã giảm giá trong tạo đơn hàng	Invoke-RestMethod	totalAmount/discountAmount tính đúng server-side; mã hết lượt → 404 kèm rollback tồn kho	PASS

Chức năng	Phương pháp	Kết quả cụ thể	Kết luận
Trang quản lý đơn hàng Admin (backend)	JWT ADMIN tự ký	State machine đúng ở mọi ca chuyển trạng thái hợp lệ/không hợp lệ; hoàn kho đúng khi huỷ	PASS
Trang quản lý đơn hàng Admin (UI)	Kiểm thử thủ công	6/6 bước (bảng, filter, search, modal, đổi trạng thái)	6/6 PASS
Nhật ký thao tác (AuditLog)	Script Node + JWT tự ký	Ghi đúng userId/action/entityId, redact mật khẩu, không ghi trên route không đánh dấu	5/5 PASS
Đăng nhập Google (chống chiếm tài khoản)	curl + kiểm thử thủ công UI	Thiếu token → 400; token giả → 401	12/12 PASS (UI)
Thanh toán VietQR thật qua ngrok	Thanh toán ngân hàng thật	Webhook xác nhận đúng, tồn kho trừ đúng lúc PAID, `/orders` cập nhật realtime	PASS
Đánh giá sản phẩm (Reviews) — API	Script Node, JWT tự ký, 2 CUSTOMER + 1 ADMIN thật	16 ca (tạo/sửa/xoá, 403 sai chủ, 409 trùng, không lộ email, ADMIN xoá được)	16/16 PASS
Đánh giá sản phẩm (Reviews) — UI	Kiểm thử thủ công	Gửi/sửa/xoá review, khách chưa đăng nhập thấy đúng lời mời đăng nhập	6/6 PASS
Chọn item giỏ hàng để thanh toán + Progress-step	Kiểm thử thủ công (người dùng theo dõi trực tiếp)	Chọn/bỏ chọn từng dòng, "Chọn tất cả", giỏ giữ đúng phần không chọn	6/6 PASS
Sổ địa chỉ — API	Script Node, JWT tự ký, 2 CUSTOMER thật	16 ca (tạo tự động mặc định, đổi mặc định, 403 sai chủ, xoá tự đề bạt mặc định mới)	16/16 PASS
Sổ địa chỉ — UI	Chưa thực hiện	7 bước đã liệt kê trong PROGRESS.md, CHƯA được kiểm thử bằng mắt	CHƯA TEST
[Đợt này] Sửa lỗi hoàn kho sai khi huỷ đơn VietQR chưa thanh toán	Script Node, JWT ADMIN tự ký, backend thật	Đơn COD trừ/hoàn kho đúng; đơn VietQR PENDING không còn bị cộng sai khi huỷ	4/4 PASS
[Đợt này] Sửa lỗi tin userId từ body client trong POST /orders	Script Node, JWT tự ký bằng JWT_SECRET thật	Body có userId → 400; cookie hợp lệ → đúng userId; không cookie → guest; cookie giả → 401	4/4 PASS

Bảng kết quả trên chỉ trích một phần tiêu biểu; chi tiết đầy đủ toàn bộ các lần kiểm thử trong suốt quá trình phát triển được ghi lại theo thời gian thực trong nhật ký phát triển của dự án (`PROGRESS.md`).

5.2.1 Danh sách lỗi thật đã phát hiện và khắc phục

Bảng dưới đây là bằng chứng cụ thể cho năng lực gỡ lỗi thực tế trong quá trình phát triển — mỗi dòng là một lỗi THẬT đã xảy ra trên hệ thống đang chạy, không phải lỗi giả định.

Bảng 3. Các lỗi thật đã phát hiện và khắc phục trong quá trình phát triển

Thời điểm	Triệu chứng / Nguyên nhân	Hậu quả nếu không sửa	Cách khắc phục
Trước 08-29	Guard đọc JWT bằng <code>`jwt.decode()`</code> không xác minh chữ ký	Ai cũng giả mạo được <code>`sub`</code> để đọc đơn hàng người khác	Chuyển sang <code>`JwtAuthGuard`/`JwtStrategy`</code> xác minh chữ ký chuẩn
Trước 08-29	Giá tiền/tổng tiền đọc trực tiếp từ request client	Có thể đặt hàng với giá tự khai	Server luôn tự tính lại từ giá thật trong DB
29/08/2026	Sửa sản phẩm có biến thể trả lỗi 400 "property id should not exist"	Không sửa được sản phẩm → không bật lại <code>`isActive`</code> → storefront trống	Map tường minh đúng 6 field DTO cho phép, không gửi nguyên object variant đọc từ GET
29/08/2026	<code>`ProductService.update()`</code> validate <code>`variants`</code> nhưng không lưu	UI báo thành công nhưng biến thể không đổi	Xử lý xoá sạch rồi tạo lại trong 1 transaction
29/08/2026	Mã giảm giá bị rơi mất giữa giỏ hàng và trang thanh toán	Giảm giá không được áp dụng khi đặt hàng thật	Bridge <code>`discountCode`</code> qua <code>sessionStorage</code> , re-validate ở trang thanh toán
29/08/2026	Race condition: 2 đơn cùng tranh lượt cuối của 1 mã giảm giá	Có thể vượt quá <code>`maxUsage`</code> đã đặt	Dùng <code>`updateMany`</code> có điều kiện <code>`usedCount < maxUsage`</code>
29/08/2026	<code>`catch`</code> trong tạo đơn chỉ re-throw <code>`BadRequestException`</code>	Lỗi mã giảm giá (404) bị nuốt thành thông báo chung chung	Đổi điều kiện sang <code>`instanceof HttpException`</code>
30/08/2026	Stale closure khi upload 2 ảnh biến thể liên tiếp	Mất 1 ảnh vừa upload cho cùng 1 màu	Tính state từ <code>`prev.variants`</code> thay vì biến đóng cũ
31/08/2026	GoogleLoginDto cho <code>`token`</code> optional, nhận <code>`email`/`fullName`</code> từ client	Bỏ qua <code>`token`</code> = bỏ qua toàn bộ xác minh chữ ký → chiếm đoạt tài khoản bất kỳ không cần mật khẩu	<code>`token`</code> bắt buộc, xoá hẳn <code>`email`/`fullName`</code> khỏi DTO — chỉ lấy từ payload đã xác minh

Thời điểm	Triệu chứng / Nguyên nhân	Hậu quả nếu không sửa	Cách khắc phục
01/09/2026	Form Hồ sơ luôn báo lỗi 400 dù dữ liệu hợp lệ	Không sửa được hồ sơ cá nhân	Gửi field thừa `avatarUrl`; đồng thời phát hiện `@IsOptional()` không bỏ qua chuỗi rỗng `""`
01/09/2026	Mã QR hiển thị lỗi (ảnh vỡ)	Khách không quét được mã thanh toán	`qrCode` từ PayOS là chuỗi EMVCo thô, không phải URL ảnh — vẽ lại bằng `qrcode.react`
02/09/2026	Webhook trả 404 khi `orderCode` không khớp đơn nào	`payos.webhooks.confirm()` từ chối đăng ký URL webhook ở CẢ production	Trả 200 kèm log cảnh báo thay vì throw 404
02/09/2026	"Hết hàng ảo": đơn VietQR bị bỏ ngang vẫn giữ tồn kho đã trừ	Sản phẩm hiển thị hết hàng dù không ai mua thật	Chuyển thời điểm trừ kho VietQR sang lúc webhook xác nhận PAID (Hướng B)
02/09/2026	`/orders/lookup` hiển thị nguyên văn enum tiếng Anh	Trải nghiệm không nhất quán ngôn ngữ	Dùng bảng nhãn tiếng Việt `SHIPPING_STATUS_LABEL`
26/09/2026 (đợt này)	Huỷ đơn VietQR chưa thanh toán vẫn cộng lại tồn kho	Tồn kho ảo cộng dồn theo từng đơn VietQR bị huỷ, có thể dẫn tới bán vượt tồn kho thật	Chỉ hoàn kho khi kho đã thực sự bị trừ trước đó
26/09/2026 (đợt này)	`POST /orders` tin `userId` do client tự khai trong body	Có thể chen đơn hàng giả vào lịch sử của người dùng khác (userId lộ qua `GET /reviews` công khai)	Thêm `OptionalJwtAuthGuard`, lấy userId từ JWT đã xác minh, xoá field khỏi DTO

5.3 Nhận xét về chất lượng kiểm thử

Điểm mạnh: các luồng nghiệp vụ quan trọng nhất (thanh toán, đơn hàng, xác thực) đều đã được kiểm thử bằng dữ liệu và điều kiện THẬT (bao gồm cả 1 giao dịch ngân hàng thật qua ngrok), không chỉ dừng ở mức giả định trên giấy.

Điểm hạn chế cần nêu trung thực: hệ thống chưa có bộ unit test/e2e test tự động chạy được trong CI, nên mọi lần thay đổi code đều phụ thuộc vào việc kiểm thử thủ công lặp lại — đây là rủi ro thực sự khi hệ thống lớn dần. Ngoài ra, số địa chỉ (Address) mới chỉ được kiểm thử đầy đủ ở tầng API, phần giao diện vẫn đang chờ kiểm thử bằng mắt tại thời điểm hoàn thành báo cáo này.

CHƯƠNG 6 — KẾT LUẬN & HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Sau quá trình thực hiện, đề án đã hoàn thành các mục tiêu đề ra ở Chương 1:

- Xây dựng hoàn chỉnh backend RESTful API với 49 endpoint trên 17 module, áp dụng đầy đủ các lớp bảo mật: xác thực JWT qua cookie HttpOnly, phân quyền RBAC, giới hạn tần suất request theo từng endpoint nhạy cảm, validate đầu vào nghiêm ngặt bằng ValidationPipe.
- Module PIM với dữ liệu chủ (danh mục, danh mục con, thương hiệu) đã được chuẩn hoá hoàn toàn thành bảng quan hệ có khoá ngoại, thay cho chuỗi tự do ở phiên bản khởi tạo ban đầu.
- Tích hợp thanh toán VietQR thật qua PayOS, đã kiểm thử thành công với giao dịch ngân hàng thật, xử lý đúng bài toán trừ tồn kho trong giao dịch bất đồng bộ.
- Xây dựng hoàn chỉnh giao diện storefront và trang quản trị, thống nhất 1 hệ thống thiết kế xuyên suốt toàn site.
- Phát hiện và khắc phục 16 lỗi thật trong quá trình phát triển (Chương 5), bao gồm 2 lỗi vừa được phát hiện và sửa ngay trong quá trình biên soạn báo cáo này — minh chứng cho việc rà soát code được thực hiện nghiêm túc tới thời điểm cuối cùng, không dừng lại khi "đủ dùng".

6.2 Hạn chế

Các hạn chế dưới đây được ghi nhận trung thực theo đúng tình trạng thật của hệ thống tại thời điểm hoàn thành báo cáo, không né tránh:

- Chưa triển khai lên môi trường production thật: chưa có tên miền và SSL riêng, chưa chọn nền tảng triển khai (VPS hay PaaS), webhook PayOS mới được đăng ký thử nghiệm qua ngrok (tạm thời, không lưu vào cấu hình chính thức).
- Chưa có cơ chế dọn dẹp ảnh mờ côi trên Cloudinary: ảnh được upload ngay khi Admin chọn file, nếu sau đó bị xoá khỏi form thì file vẫn còn tồn trên Cloudinary, chưa có script quét dọn định kỳ.

- Trang chủ và trang danh mục phụ kiện chưa phân trang phía server thật — đang tải tối đa 100 sản phẩm rồi lọc ở trình duyệt, sẽ không còn phù hợp khi catalog vượt quá con số này.
- Chưa có index cơ sở dữ liệu trên các cột thường dùng để sắp xếp/tìm kiếm (``createdAt``, ``isActive``, ``price``, ``title``) — chưa ảnh hưởng ở quy mô dữ liệu hiện tại nhưng sẽ cần bổ sung khi catalog lớn.
- Sentry đã được khởi tạo (``Sentry.init()``) nhưng CHƯA được nối vào bộ lọc lỗi toàn cục — lỗi 500 thật hiện tại chưa thực sự được gửi về Sentry để theo dõi tập trung.
- Chưa có tích hợp hoàn tiền tự động qua PayOS: huỷ một đơn đã thanh toán chỉ hiển thị cảnh báo, Admin phải tự hoàn tiền thủ công qua PayOS Dashboard.
- Còn tồn tại race condition ở bước trừ tồn kho cho đơn COD (đọc giá trị rồi ghi đè, chưa dùng cập nhật có điều kiện như cơ chế đã áp dụng cho mã giảm giá) — rủi ro thấp ở quy mô hiện tại nhưng cần khắc phục trước khi vận hành thật với lượng đơn đồng thời cao.
- AuditLog hiện chỉ ghi mà chưa có API hay giao diện để Admin xem lại nhật ký đã ghi.
- Chưa xây dựng bộ unit test/e2e test tự động; toàn bộ kiểm thử hiện dựa vào script gọi API thủ công và kiểm thử giao diện bằng tay.
- Vai trò VENDOR mới dừng ở mức được cấp quyền upload ảnh, chưa có luồng nghiệp vụ riêng (quản lý sản phẩm/đơn hàng của chính đối tác đó).
- Số địa chỉ (Address) đã kiểm thử đầy đủ ở tầng API nhưng phần giao diện chưa được kiểm thử bằng mắt tại thời điểm hoàn thành báo cáo.

6.3 Hướng phát triển

- Triển khai chính thức lên môi trường production: chọn nền tảng hạ tầng, cấu hình tên miền/SSL, đăng ký webhook PayOS với URL thật, tách biệt cấu hình development/production.
- Bổ sung ``minPrice``/``maxPrice`` vào bộ lọc sản phẩm để có thể chuyển trang chủ/danh mục sang phân trang phía server hoàn chỉnh.

- Viết script/cron job dọn ảnh Cloudinary mờ côi, định kỳ đối chiếu ảnh đang lưu với URL thực tế được tham chiếu trong cơ sở dữ liệu.
- Bổ sung index cơ sở dữ liệu cho các cột dùng để sắp xếp/tìm kiếm khi catalog mở rộng.
- Hoàn thiện tích hợp Sentry (nối vào bộ lọc lỗi toàn cục) và cân nhắc tích hợp hoàn tiền tự động qua API của PayOS.
- Đổi cơ chế trừ tồn kho cho đơn COD sang cập nhật có điều kiện (theo đúng mẫu đã áp dụng cho mã giảm giá) để loại bỏ hoàn toàn race condition.
- Xây dựng giao diện xem AuditLog cho Admin, và bổ sung cơ chế phát hiện tái sử dụng refresh token đã bị thu hồi (reuse detection) để tăng cường phát hiện token bị đánh cắp.
- Xây dựng bộ test tự động (unit test cho service nghiệp vụ, e2e test cho các luồng quan trọng) tích hợp vào quy trình CI/CD.
- Phát triển đầy đủ luồng nghiệp vụ cho vai trò VENDOR — cho phép đối tác tự quản lý sản phẩm và đơn hàng của mình trong phạm vi được cấp phép.

TÀI LIỆU THAM KHẢO

1. NestJS Documentation. <https://docs.nestjs.com>
2. Prisma Documentation. <https://www.prisma.io/docs>
3. Next.js Documentation (App Router). <https://nextjs.org/docs>
4. PayOS Developer Documentation. <https://payos.vn/docs>
5. Cloudinary Documentation. <https://cloudinary.com/documentation>
6. Upstash Redis Documentation. <https://upstash.com/docs/redis>
7. Neon (Serverless Postgres) Documentation. <https://neon.tech/docs>
8. Google Identity Services — Sign In With Google.
<https://developers.google.com/identity>
9. OWASP Top 10. <https://owasp.org/www-project-top-ten/>
10. JSON Web Token (RFC 7519). <https://datatracker.ietf.org/doc/html/rfc7519>

PHỤ LỤC A — DANH SÁCH ĐẦY ĐỦ API ENDPOINT

* Route `POST /orders` áp dụng `OptionalJwtAuthGuard`: nếu request có cookie đăng nhập hợp lệ, hệ thống tự gắn đơn hàng cho đúng người dùng đó; nếu không có cookie, xử lý như khách vãng lai. Đây là kết quả sau khi khắc phục lỗi hồng tin `userId` từ body client, được trình bày chi tiết ở Chương 5.

#	Method	Endpoint	Guard / Quyền	Mô tả
1	POST	/auth/register	—	Đăng ký tài khoản (3/phút)
2	POST	/auth/login	—	Đăng nhập email/mật khẩu (5/phút)
3	POST	/auth/google	—	Đăng nhập bằng Google (5/phút)
4	POST	/auth/refresh	—	Xoay vòng cập token (10/phút)
5	POST	/auth/logout	—	Thu hồi refresh token
6	GET	/auth/me	JWT	Thông tin người dùng hiện tại
7	POST	/orders	JWT tùy chọn *	Tạo đơn hàng (khách vãng lai hoặc đã đăng nhập)
8	GET	/orders/my-orders	JWT	Đơn hàng của chính mình
9	GET	/orders/user/:userId	JWT + ADMIN	Đơn hàng của 1 người dùng bất kỳ
10	POST	/orders/lookup	—	Tra cứu đơn hàng bằng mã đơn + SĐT (5/phút)
11	GET	/orders/admin/all	JWT + ADMIN	Danh sách đơn hàng có phân trang/lọc
12	PATCH	/orders/:id/shipping-status	JWT + ADMIN	Đổi trạng thái vận chuyển (state machine)
13	GET	/orders/:id/status	—	Chỉ trả paymentStatus (dùng cho polling)
14	GET	/orders/:id	JWT (chủ đơn/ADMIN)	Chi tiết 1 đơn hàng
15	POST	/payments/create-qr	—	Tạo link/QR thanh toán PayOS

#	Method	Endpoint	Guard / Quyền	Mô tả
16	POST	/payments/payos-webhook	— (xác minh chữ ký)	Webhook PayOS xác nhận thanh toán
17	GET	/products	—	Danh sách sản phẩm công khai (phân trang)
18	GET	/products/categories	—	Danh sách danh mục (có cache)
19	GET	/products/admin/all	JWT + ADMIN	Danh sách sản phẩm cho Admin (gồm cả ẩn)
20	GET	/products/:slug	—	Chi tiết sản phẩm theo slug
21	POST	/products	JWT + ADMIN	Tạo sản phẩm mới
22	PATCH	/products/:id	JWT + ADMIN	Cập nhật sản phẩm
23	DELETE	/products/:id	JWT + ADMIN	Xoá (mềm/cứng tùy điều kiện) sản phẩm
24	POST	/upload	JWT + ADMIN/VENDOR	Upload ảnh lên Cloudinary (20/phút)
25	POST	/ai/chat	—	Trợ lý AI tư vấn sản phẩm (10/phút)
26	GET	/reviews	—	Danh sách đánh giá + điểm trung bình theo sản phẩm
27	GET	/reviews/eligibility	JWT	Kiểm tra điều kiện được phép đánh giá
28	POST	/reviews	JWT	Gửi đánh giá (5/phút)
29	PATCH	/reviews/:id	JWT (chủ đánh giá)	Sửa đánh giá của chính mình
30	DELETE	/reviews/:id	JWT (chủ hoặc ADMIN)	Xoá đánh giá
31	GET	/addresses	JWT	Danh sách địa chỉ của chính mình
32	POST	/addresses	JWT	Thêm địa chỉ mới
33	PATCH	/addresses/:id	JWT (chủ sở hữu)	Sửa địa chỉ
34	PATCH	/addresses/:id/set-default	JWT (chủ sở hữu)	Đặt làm địa chỉ mặc định
35	DELETE	/addresses/:id	JWT (chủ sở hữu)	Xoá địa chỉ
36	GET	/discounts/code/:code	—	Kiểm tra hiệu lực mã giảm giá

#	Method	Endpoint	Guard / Quyền	Mô tả
37	GET	/brands	—	Danh sách thương hiệu (có cache)
38	POST	/brands	JWT + ADMIN	Tạo thương hiệu
39	DELETE	/brands/:id	JWT + ADMIN	Xoá thương hiệu (nếu không còn sản phẩm)
40	GET	/sub-categories	—	Danh sách danh mục con
41	POST	/sub-categories	JWT + ADMIN	Tạo danh mục con
42	DELETE	/sub-categories/:id	JWT + ADMIN	Xoá danh mục con (nếu không còn sản phẩm)
43	POST	/categories	JWT + ADMIN	Tạo danh mục chính
44	DELETE	/categories/:id	JWT + ADMIN	Xoá danh mục chính (nếu không còn sản phẩm)
45	GET	/users/:id	JWT (chủ hoặc ADMIN)	Hồ sơ người dùng (không kèm mật khẩu)
46	PATCH	/users/:id	JWT (chủ hoặc ADMIN)	Cập nhật hồ sơ
47	PATCH	/users/:id/password	JWT (chủ hoặc ADMIN)	Đổi mật khẩu
48	PATCH	/users/:id/avatar	JWT (chủ hoặc ADMIN)	Cập nhật ảnh đại diện (tối đa 5MB)
49	GET	/health	—	Kiểm tra sức khoẻ hệ thống (loại trừ khỏi rate-limit)

Ngoài các endpoint API trên, hệ thống còn phục vụ file tĩnh qua `GET /data/\*` (dữ liệu mẫu sản phẩm dùng nội bộ cho quá trình phát triển).